



Data Structures

CPT102 – Lecture 9
Erick Purwanto



Xi'an Jiaotong-Liverpool University

西交利物浦大學

CPT102 Data Structures

Lecture 9

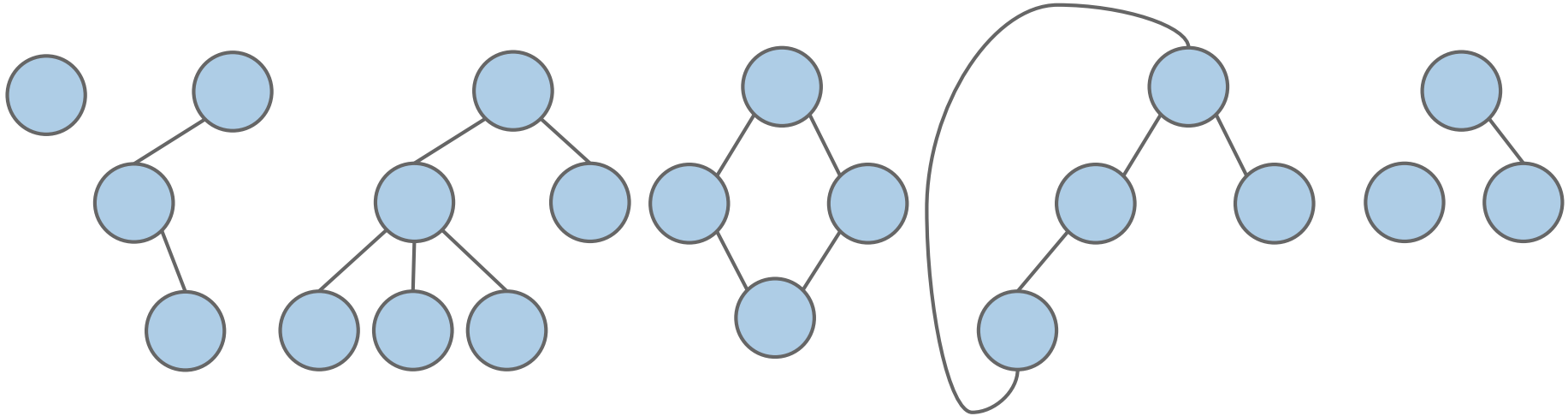
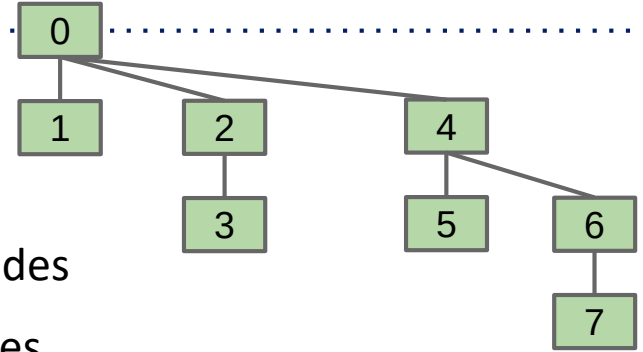
Introduction to Graphs

Welcome !

- Welcome to Lecture 9 !
- In this lecture we are going to
 - learn about trees:
 - tree traversals:
level order, depth first order (preorder, inorder, postorder)
 - learn about graphs:
 - graph problems:
connectivity, finding paths
 - graph traversals:
DFS (preorder, postorder) and BFS
 - learn about graph representations:
 - adjacency matrix, edge set, adjacency list

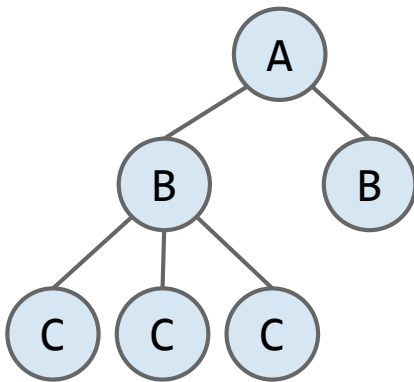
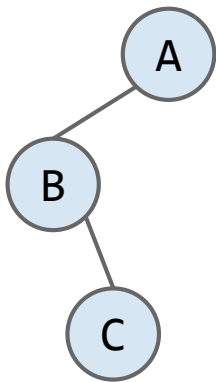
Tree Data Structure

- We have seen a tree data structure in **Lecture 8a**
- A **tree**:
 - a set of **nodes** and a set of **edges** connecting nodes
 - there is **exactly one path** between any two nodes



Rooted Tree

- Rooted tree: a tree where we've chosen one node as the **root**
 - every node N except the root has ***exactly one* parent**, defined as the first node on the path from N to the root
 - a node with no **child** is called a **leaf**



- A:
- B:
- B:
- C:

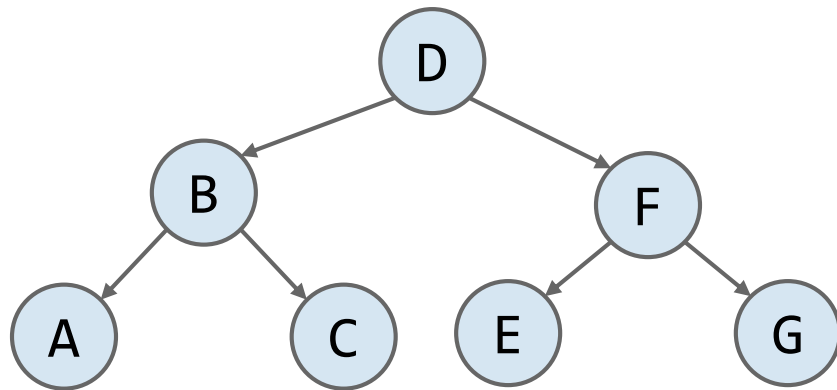
Rooted Tree

- A Java implementation: (BST: *Binary Search Tree*, more on it later in the course!)

```
public class BST<Key extends Comparable<Key>, Value> {  
    private Node root;                // root of BST  
  
    private class Node {  
        private Key key;              // sorted by key  
        private Value val;            // associated data  
        private Node left, right;     // left and right subtrees  
        private int size;             // number of nodes in subtree  
  
        public Node(Key key, Value val, int size) {  
            this.key = key;  
            this.val = val;  
            this.size = size;  
        }  
    }  
}
```

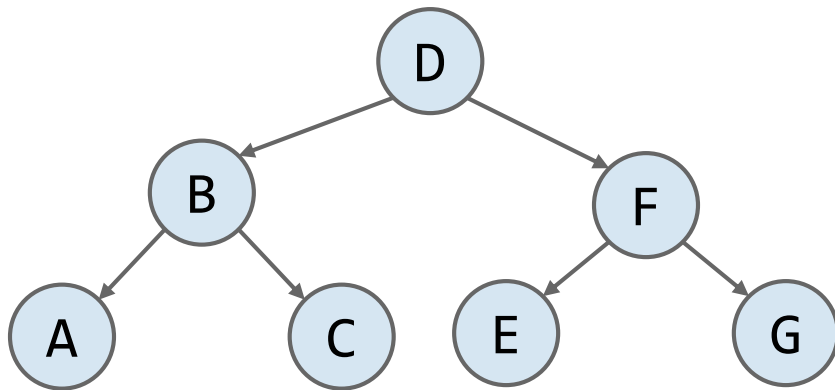
Tree Traversal and Level Order

- Sometimes we want to iterate over a tree, we call this **tree traversal**
 - there are many orders in which we might **visit** the nodes
- **Level Order**: visit top-to-bottom, left-to-right



Depth First Traversals

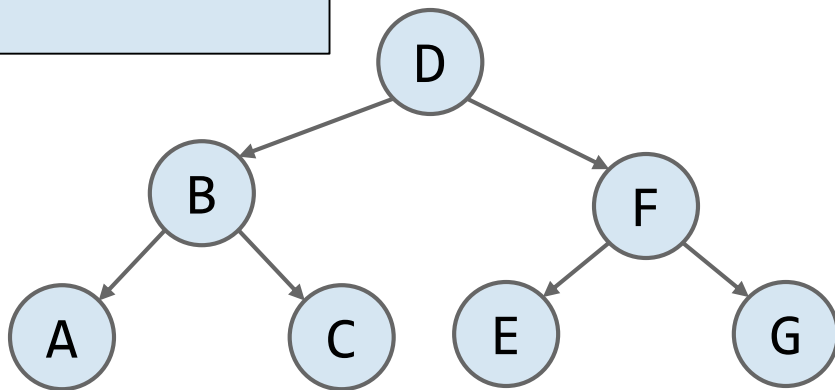
- Three types of **depth first traversal**: preorder, inorder, postorder
 - basic idea: traverse the deep nodes before the shallow ones
 - **visit**: access the content (e.g., print the key) of a node
 - **traverse**: explore a node without accessing the content



Preorder Traversal

- **Preorder:** visit a node, then traverse left child and traverse right child
 - defined recursively

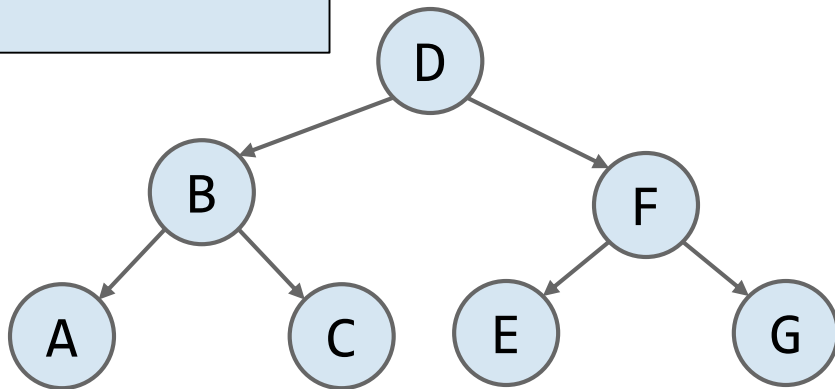
```
preorder(Node x) {  
    if (x == null) return;  
    print(x.key)  
    preorder(x.left)  
    preorder(x.right)  
}
```



Inorder Traversal

- **Inorder**: traverse left child, visit a node, then traverse right child
 - defined recursively

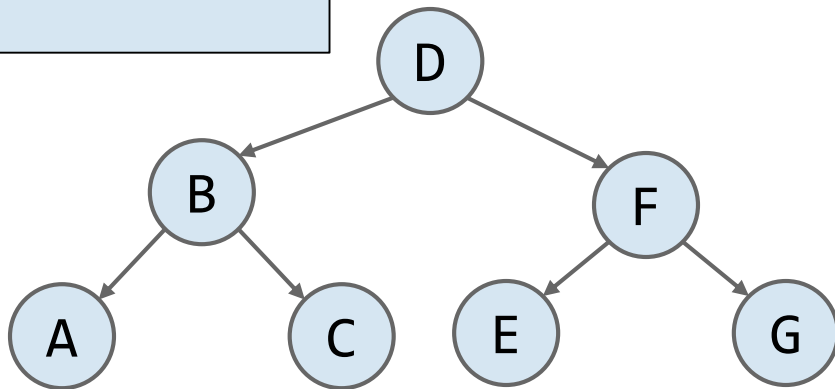
```
inorder(Node x) {  
    if (x == null) return;  
    inorder(x.left)  
    print(x.key)  
    inorder(x.right)  
}
```



Postorder Traversal

- **Postorder**: traverse left child, traverse right child, then visit a node
 - defined recursively

```
postorder(Node x) {  
    if (x == null) return;  
    postorder(x.left)  
    postorder(x.right)  
    print(x.key)  
}
```



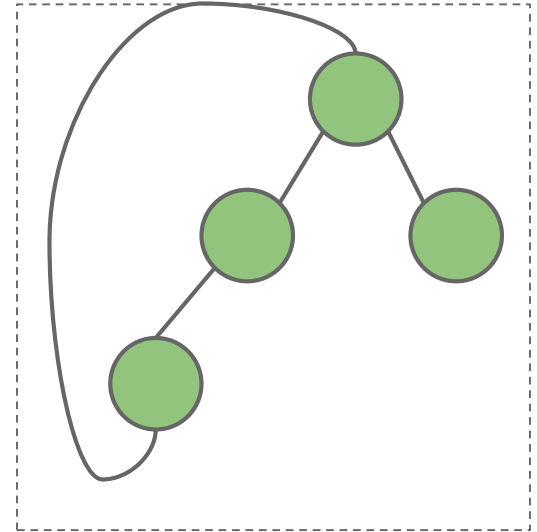
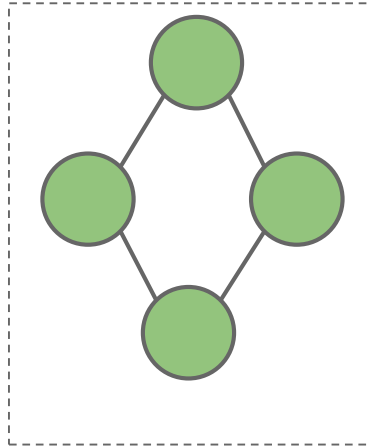
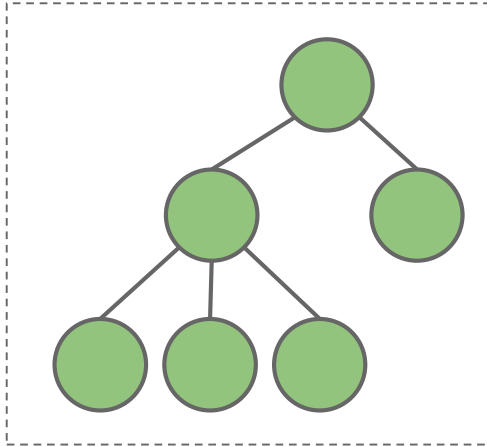
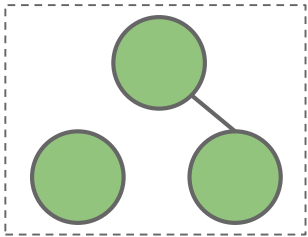
Trees and Hierarchical Relationships

- Trees are good for representing strict hierarchical relationships
 - but not every relationship is hierarchical
 - for example, Suzhou Metro
 - contains cycles,
more than one way to get from A to B
- We generalize to graph



Graph Data Structure

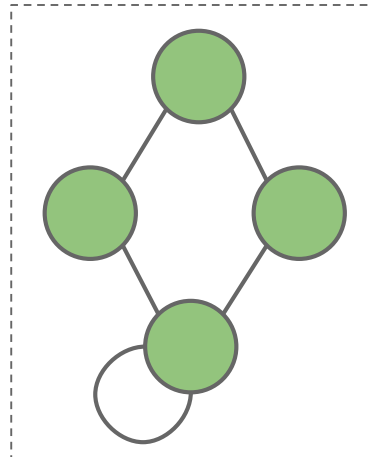
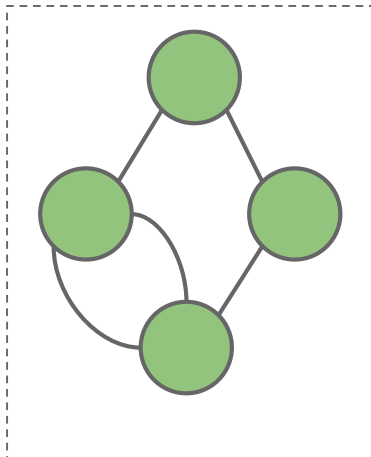
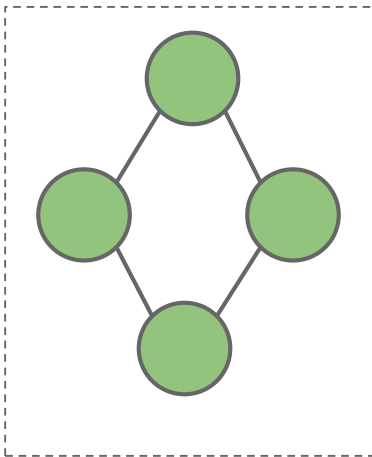
- A **graph** consists of:
 - a set of nodes
 - a set of zero or more edges, each of which connects two nodes
- Which of the following are graphs?



Simple Graph

- A **simple graph** is a graph with:
 - no edges that connect a vertex to itself, i.e. **no loops**
 - no two edges that connect the same vertices, i.e. **no parallel edges**
- Which of the following are simple graphs?

in CPT102, unless otherwise stated, *all graphs are simple*

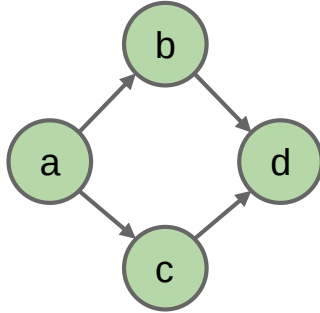


Graph Types

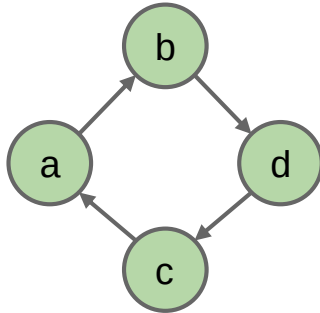
- Based on directionality, or whether there are cycles:

Directed:

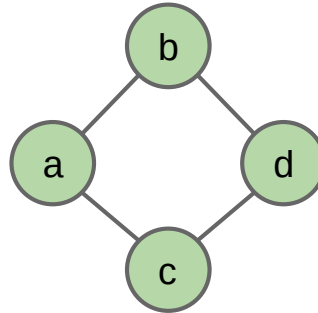
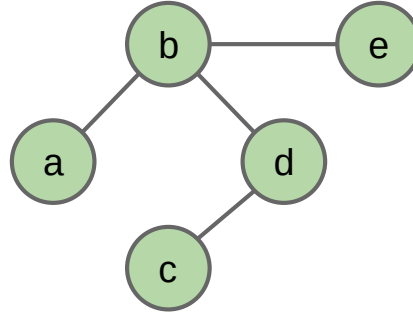
Acyclic:



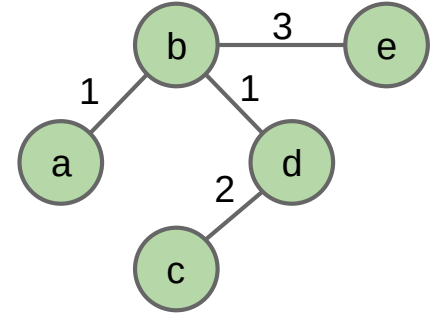
Cyclic:



Undirected:

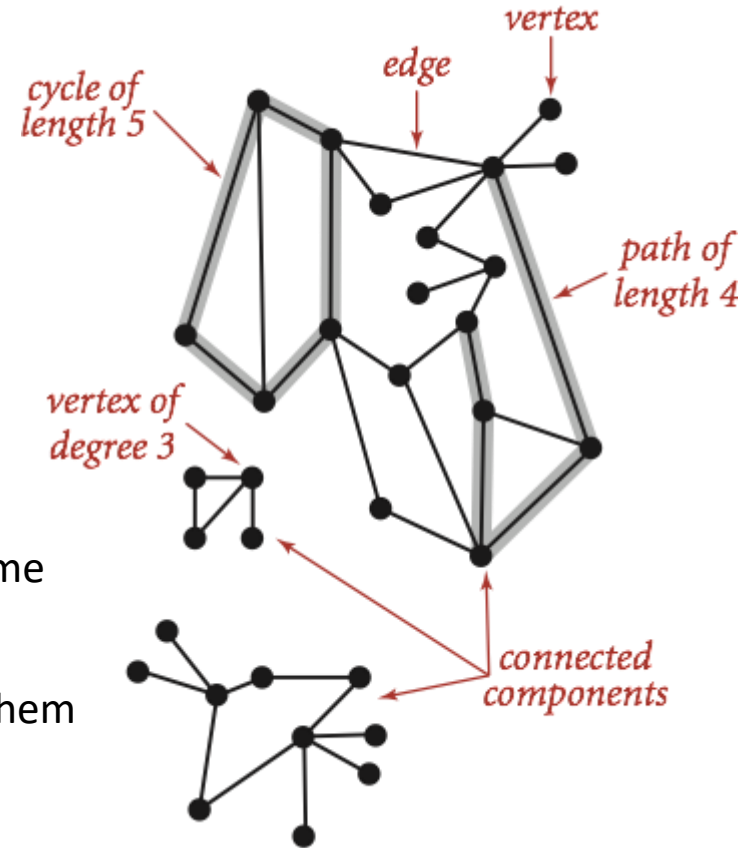


or, Weighted:
(with edge labels)



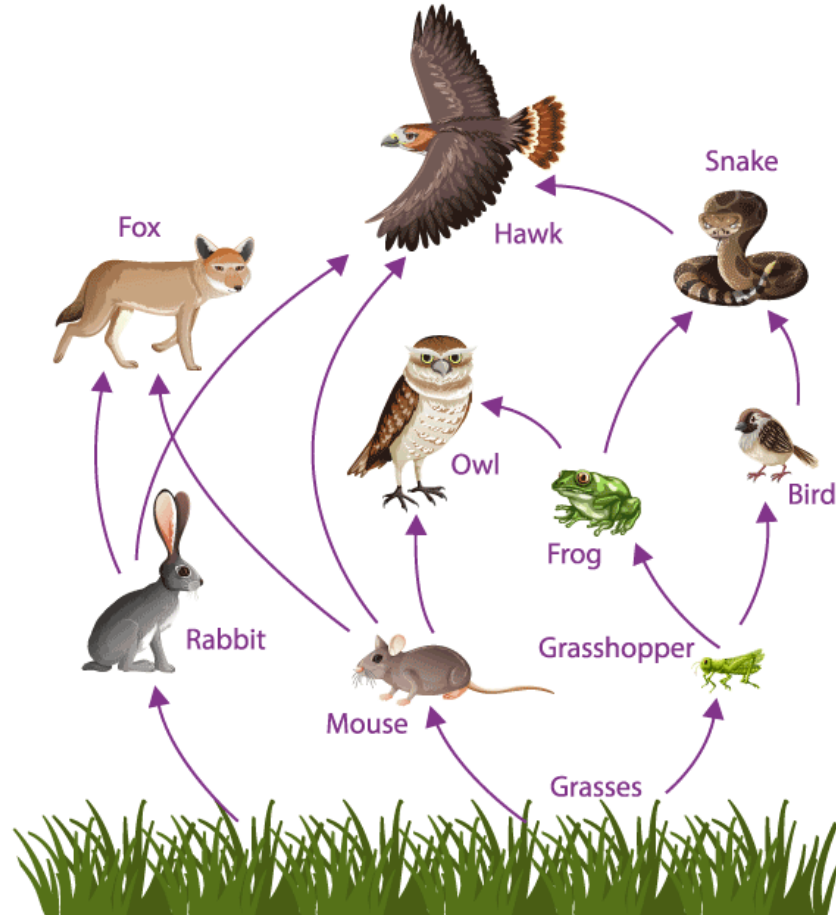
Graph Terminologies

- Graph:
 - set of **vertices** (*nodes*)
 - set of **edges** (pairs of vertices)
 - vertices with an edge between are **adjacent**
 - optional: vertices or edges may have **labels** (or **weights**)
- A **path** is a sequence of vertices connected by edges
 - a **simple path** is a path without repeated vertices
- A **cycle** is a path whose first and last vertices are the same
 - a graph with a cycle is **cyclic**
- Two vertices are **connected** if there is a path between them
 - if all vertices are connected, we say the graph is **connected**



Directed Graph Example

- Food Web



.....

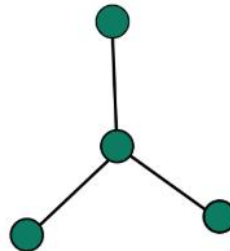
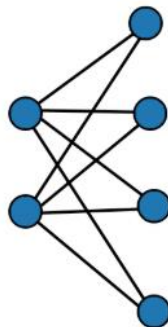
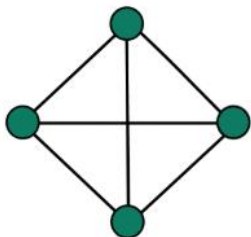
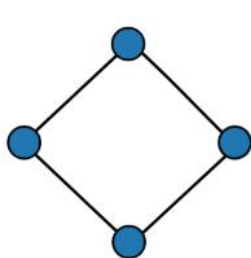
-

Graph Problems

- Some well known **graph problems** and their common names:
 - **s-t Path**. Is there a path between vertices s and t ?
 - **Connectivity**. Is the graph connected, i.e. is there a path between all vertices?
 - **Biconnectivity**. Is there a vertex whose removal disconnects the graph?
 - **Shortest s-t Path**. What is the shortest path between vertices s and t ?
 - **Cycle Detection**. Does the graph contain any cycles?
 - **Euler Tour**. Is there a cycle that uses every edge exactly once?
 - **Hamilton Tour**. Is there a cycle that uses every vertex exactly once?
 - **Planarity**. Can you draw the graph on paper with no crossing edges?
 - **Isomorphism**. Are two graphs isomorphic (the same graph in disguise)?

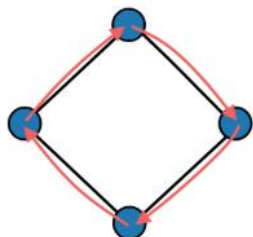
Graph Problem Difficulty

- The problems may seem similar:
 - **Euler Tour.** Is there a cycle that uses every edge exactly once?
 - **Hamilton Tour.** Is there a cycle that uses every vertex exactly once?

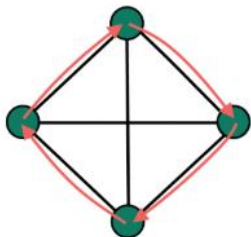


Graph Problem Difficulty

- The problems may seem similar:
 - **Euler Tour.** Is there a cycle that uses every edge exactly once?
 - **Hamilton Tour.** Is there a cycle that uses every vertex exactly once?



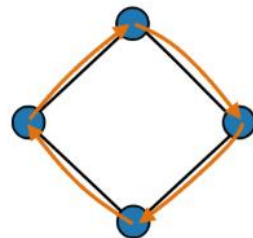
Hamiltonian



Hamiltonian

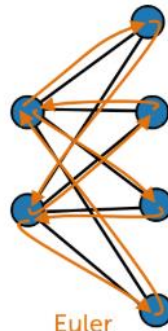
Not Hamiltonian

Not Hamiltonian



Euler

Not Euler



Euler

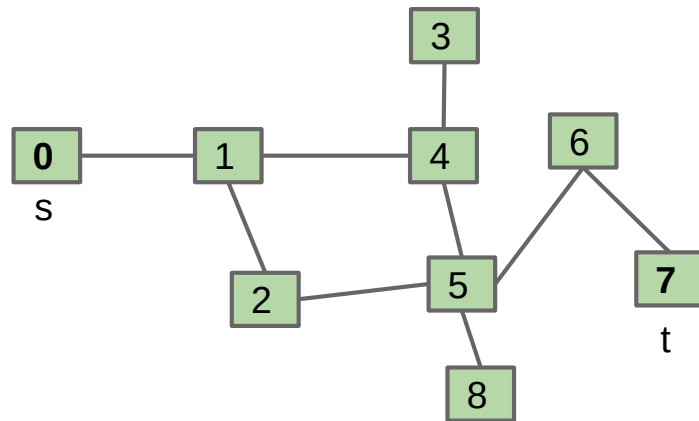
Not Euler

Graph Problem Difficulty

- The problems may seem similar:
 - **Euler Tour**. Is there a cycle that uses every edge exactly once?
 - **Hamilton Tour**. Is there a cycle that uses every vertex exactly once?
- Can we solve them efficiently?
 - an **efficient** Euler tour algorithm $O(\text{\#edges})$ was found as early as 1873
 - despite decades of intense study, **no efficient algorithm** for a Hamilton tour exists, and best known algorithms are **exponential time**
 - in fact, if you could solve this (or show that there is no such efficient algorithms), you win 1M USD for solving one of the **Millennium Prize Problems**

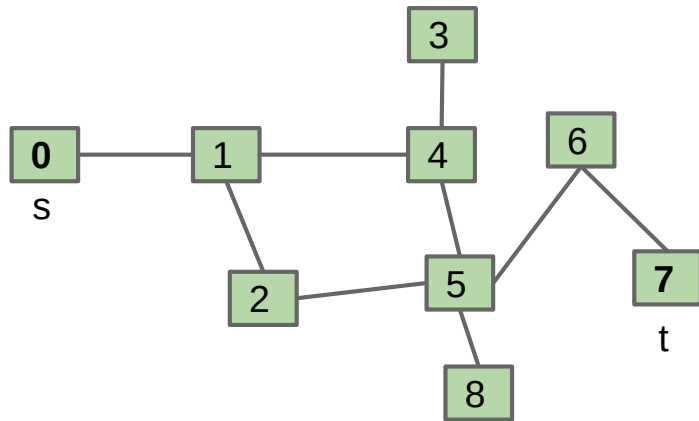
s-t Connectivity

- Let's solve one of the graph problems called the **s-t connectivity** problem
 - given **source** vertex s and a **target** vertex t , is there a **path** between s and t ?
- Requires us to traverse the graph somehow
 - need to come up with an algorithm for **connected(s, t)**



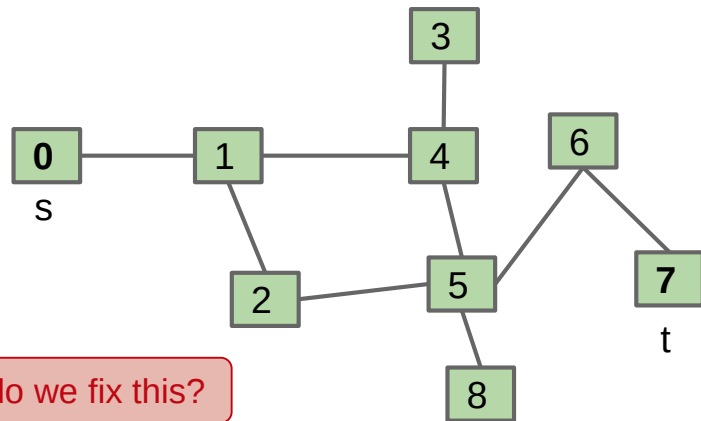
First s-t Connectivity Algorithm

- Recursive algorithm for `connected(s, t)`:
 - does `s == t`? if so, return true
 - otherwise, if `connected(v, t)` for any neighbor `v` of `s`, return true
 - return false



Bug in First s-t Connectivity Algorithm

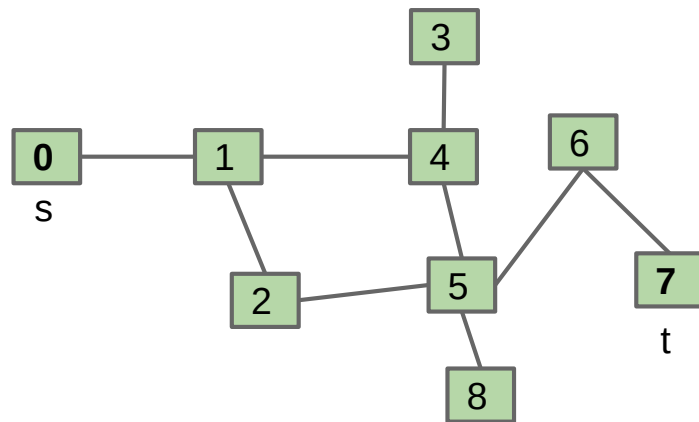
- Recursive algorithm for `connected(s, t)`:
 - does `s == t`? if so, return true
 - otherwise, if `connected(v, t)` for any neighbor `v` of `s`, return true
 - return false
- Can get caught in infinite loop, for example:
 - `connected(0, 7)`:
 - does `0 == 7`? no
 - if (`connected(1, 7)`) return true;
 - `connected(1, 7)`:
 - does `1 == 7`? no
 - if (`connected(0, 7)`) ... ← infinite loop



how do we fix this?

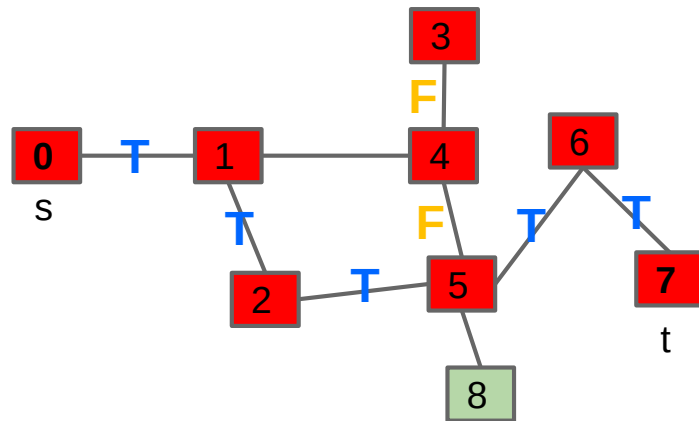
s-t Connectivity Algorithm

- `connected(s, t)`.
 - **mark s**
 - does `s == t`? if so, return true
 - otherwise, if `connected(v, t)` for any **unmarked** neighbor `v` of `s`, return true
 - return false



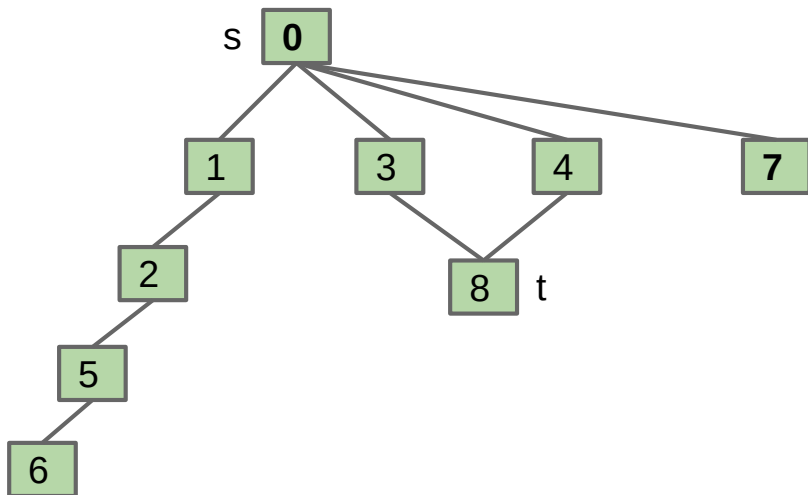
s-t Connectivity Algorithm

- `connected(s, t)`.
 - **mark s** 
 - does `s == t`? if so, return true
 - otherwise, if `connected(v, t)` for any **unmarked** neighbor `v` of `s`, return true
 - return false
- Assume using the order of values:



Depth First Traversal

- This idea of exploring a neighbor's entire subgraph before moving on to the next neighbor is known as **Depth First Traversal**
 - example: explore 1's subgraph completely before traversing the edge 0-3
 - called **depth first** because you go as deep as possible



Depth First Search

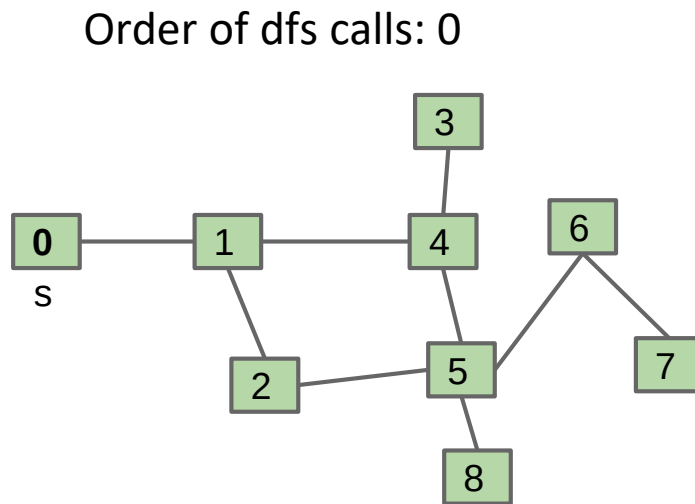
- DFS is a very powerful technique that can be used for many types of graph problems
- Let's discuss an algorithm that computes a **path** to ***every*** vertex
 - call this algorithm **DepthFirstPaths**

DepthFirstPaths Demo

Goal: Find a path from s to every other reachable vertex, visiting each vertex at most once. $\text{dfs}(v)$ is as follows:

- Mark v
- For each unmarked adjacent vertex w :
 - set $\text{edgeTo}[w] = v$
 - $\text{dfs}(w)$

#	marked	edgeTo	
0	F	-	Start by calling $\text{dfs}(0)$.
1	F	-	
2	F	-	
3	F	-	
4	F	-	
5	F	-	
6	F	-	
7	F	-	
8	F	-	



Order of dfs returns:

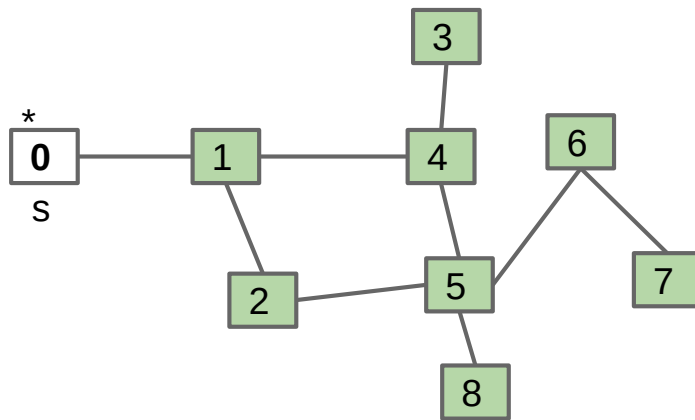
DepthFirstPaths Demo

Goal: Find a path from s to every other reachable vertex, visiting each vertex at most once. $\text{dfs}(v)$ is as follows:

- Mark v
- For each unmarked adjacent vertex w :
 - set $\text{edgeTo}[w] = v$
 - $\text{dfs}(w)$

#	marked	edgeTo	dfs(0):
0	T	-	mark(0).
1	F	0	
2	F	-	isMarked(1)? No.
3	F	-	● $\text{edgeTo}[1] = 0$. dfs(1) .
4	F	-	
5	F	-	
6	F	-	
7	F	-	
8	F	-	

Order of dfs calls: 01



Order of dfs returns:

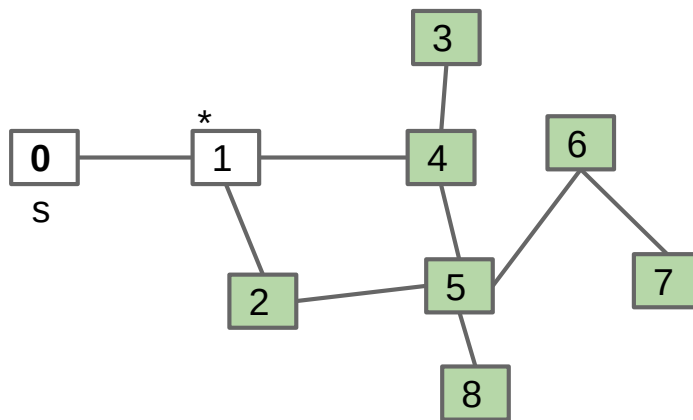
DepthFirstPaths Demo

Goal: Find a path from s to every other reachable vertex, visiting each vertex at most once. $\text{dfs}(v)$ is as follows:

- Mark v
- For each unmarked adjacent vertex w :
 - set $\text{edgeTo}[w] = v$
 - $\text{dfs}(w)$

#	marked	edgeTo	dfs(1):
0	T	-	mark(1).
1	T	0	
2	F	1	isMarked(0)? Yes.
3	F	-	isMarked(2)?
4	F	-	● edgeTo[2] = 1. dfs(2).
5	F	-	
6	F	-	
7	F	-	
8	F	-	

Order of dfs calls: 012



Order of dfs returns:

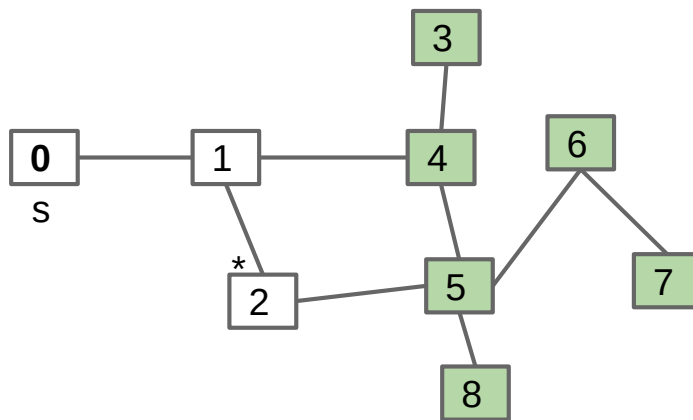
DepthFirstPaths Demo

Goal: Find a path from s to every other reachable vertex, visiting each vertex at most once. $\text{dfs}(v)$ is as follows:

- Mark v
- For each unmarked adjacent vertex w :
 - set $\text{edgeTo}[w] = v$
 - $\text{dfs}(w)$

#	marked	edgeTo	dfs(2):
0	T	-	mark(2).
1	T	0	
2	T	1	isMarked(1)? Yes.
3	F	-	isMarked(5)?
4	F	-	● edgeTo[5] = 2. dfs(5).
5	F	2	
6	F	-	
7	F	-	
8	F	-	

Order of dfs calls: 0125



Order of dfs returns:

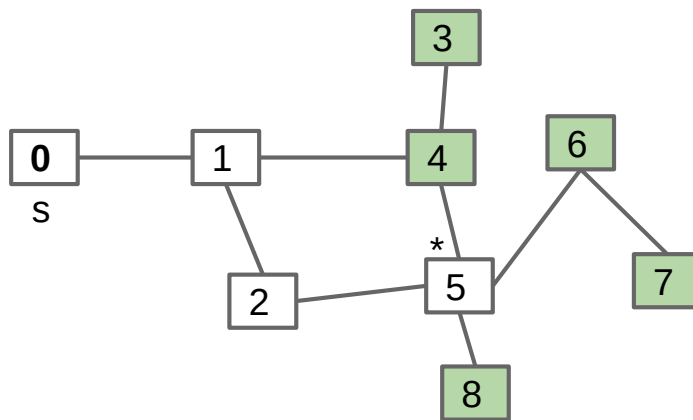
DepthFirstPaths Demo

Goal: Find a path from s to every other reachable vertex, visiting each vertex at most once. $\text{dfs}(v)$ is as follows:

- Mark v
- For each unmarked adjacent vertex w :
 - set $\text{edgeTo}[w] = v$
 - $\text{dfs}(w)$

#	marked	edgeTo	dfs(5):
0	T	-	mark(5).
1	T	0	
2	T	1	isMarked(2)? Yes.
3	F	-	isMarked(4)?
4	F	5	● edgeTo[4] = 5. dfs(4).
5	T	2	
6	F	-	
7	F	-	
8	F	-	

Order of dfs calls: 01254



Order of dfs returns:

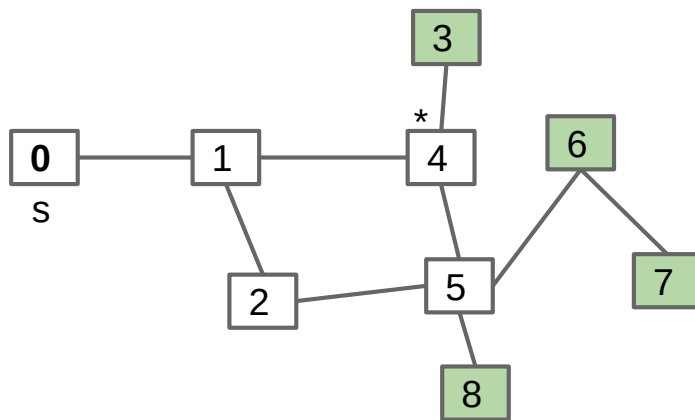
DepthFirstPaths Demo

Goal: Find a path from s to every other reachable vertex, visiting each vertex at most once. $\text{dfs}(v)$ is as follows:

- Mark v
- For each unmarked adjacent vertex w :
 - set $\text{edgeTo}[w] = v$
 - $\text{dfs}(w)$

#	marked	edgeTo	dfs(4):
0	T	-	mark(4).
1	T	0	
2	T	1	isMarked(1)? No.
3	F	4	
4	T	5	isMarked(3)? No. ● $\text{edgeTo}[3] = 4$. dfs(3).
5	T	2	
6	F	-	
7	F	-	
8	F	-	

Order of dfs calls: 012543



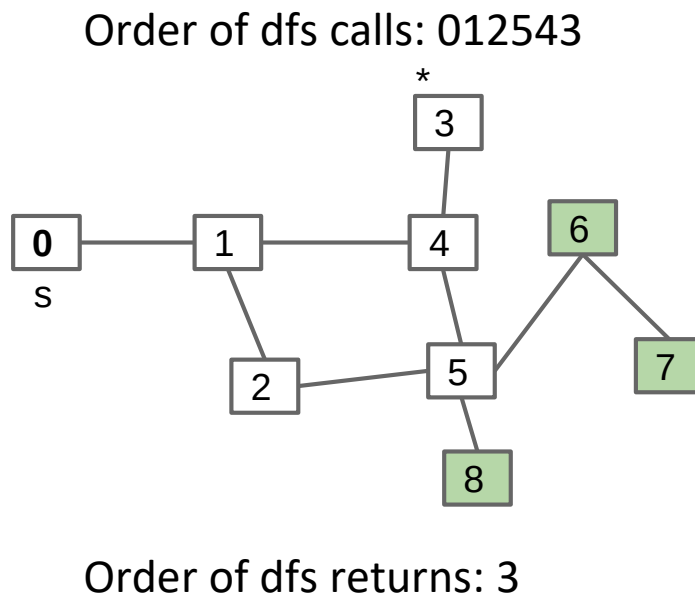
Order of dfs returns:

DepthFirstPaths Demo

Goal: Find a path from s to every other reachable vertex, visiting each vertex at most once. $\text{dfs}(v)$ is as follows:

- Mark v
- For each unmarked adjacent vertex w :
 - set $\text{edgeTo}[w] = v$
 - $\text{dfs}(w)$

#	marked	edgeTo	dfs(3):
0	T	-	mark(3).
1	T	0	
2	T	1	
3	T	4	isMarked(4)? Yes.
4	T	5	No more children! Return.
5	T	2	
6	F	-	
7	F	-	
8	F	-	



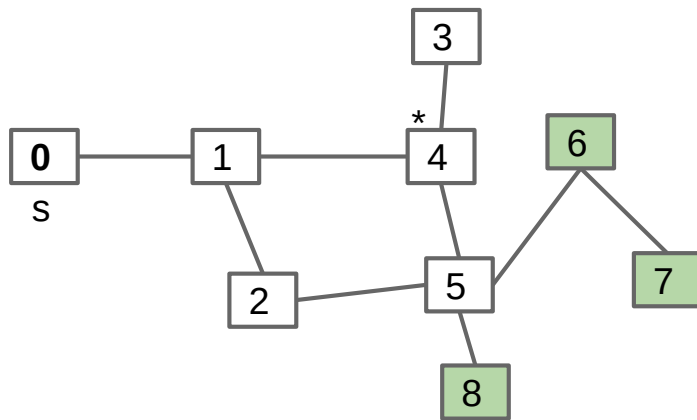
DepthFirstPaths Demo

Goal: Find a path from s to every other reachable vertex, visiting each vertex at most once. $\text{dfs}(v)$ is as follows:

- Mark v
- For each unmarked adjacent vertex w :
 - set $\text{edgeTo}[w] = v$
 - $\text{dfs}(w)$

#	marked	edgeTo	dfs(4):
0	T	-	mark(4).
1	T	0	
2	T	1	isMarked(3)? No.
3	T	4	● $\text{edgeTo}[3] = 4$. $\text{dfs}(3)$.
4	T	5	
5	T	2	No more children, return.
6	F	-	
7	F	-	
8	F	-	

Order of dfs calls: 012543



Order of dfs returns: 34

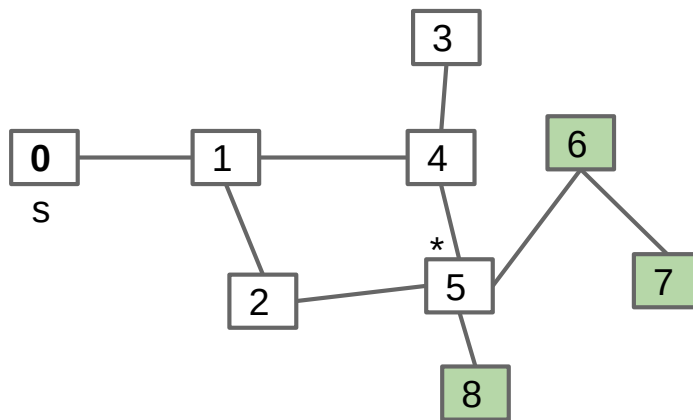
DepthFirstPaths Demo

Goal: Find a path from s to every other reachable vertex, visiting each vertex at most once. $\text{dfs}(v)$ is as follows:

- Mark v
- For each unmarked adjacent vertex w :
 - set $\text{edgeTo}[w] = v$
 - $\text{dfs}(w)$

#	marked	edgeTo	dfs(5):
0	T	-	mark(5).
1	T	0	
2	T	1	isMarked(2)? Yes.
3	T	4	isMarked(4)?
4	T	5	● edgeTo[3] = 4. dfs(4).
5	T	2	
6	F	5	isMarked(6)?
7	F	-	● edgeTo[6] = 5. dfs(6).
8	F	-	

Order of dfs calls: 0125436



Order of dfs returns: 34

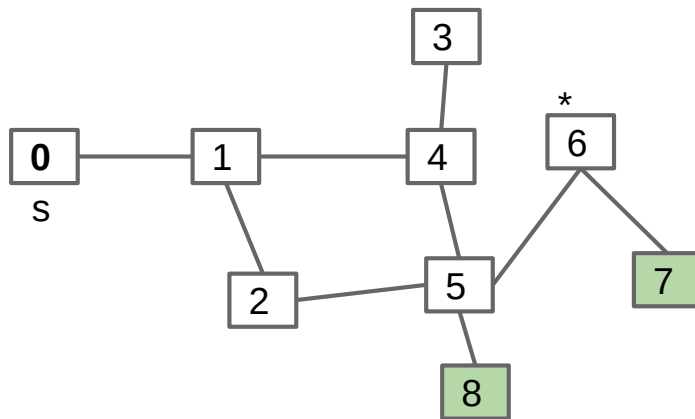
DepthFirstPaths Demo

Goal: Find a path from s to every other reachable vertex, visiting each vertex at most once. $\text{dfs}(v)$ is as follows:

- Mark v
- For each unmarked adjacent vertex w :
 - set $\text{edgeTo}[w] = v$
 - $\text{dfs}(w)$

#	marked	edgeTo	dfs(6):
0	T	-	mark(6).
1	T	0	
2	T	1	isMarked(5)? Yes.
3	T	4	isMarked(7)? No.
4	T	5	● edgeTo[7] = 6. dfs(7).
5	T	2	
6	T	5	
7	F	6	
8	F	-	

Order of dfs calls: 01254367



Order of dfs returns: 34

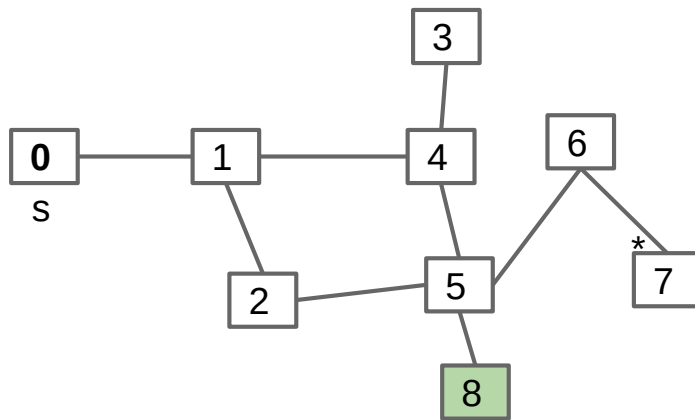
DepthFirstPaths Demo

Goal: Find a path from s to every other reachable vertex, visiting each vertex at most once. $\text{dfs}(v)$ is as follows:

- Mark v
- For each unmarked adjacent vertex w :
 - set $\text{edgeTo}[w] = v$
 - $\text{dfs}(w)$

#	marked	edgeTo	dfs(7):
0	T	-	mark(7).
1	T	0	
2	T	1	isMarked(6)? Yes.
3	T	4	
4	T	5	No more children, so return.
5	T	2	
6	T	5	
7	<u>T</u>	6	
8	F	-	

Order of dfs calls: 01254367



Order of dfs returns: 347

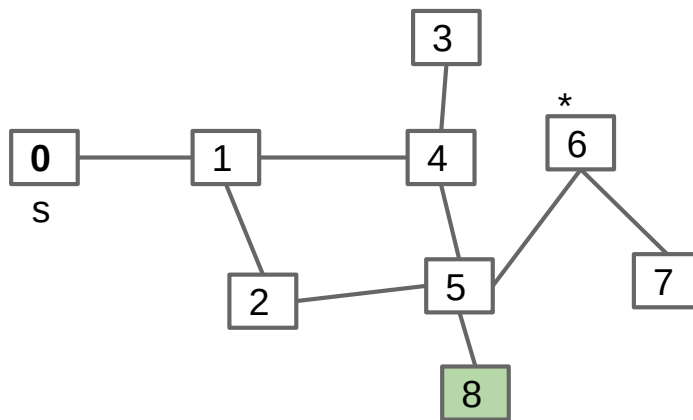
DepthFirstPaths Demo

Goal: Find a path from s to every other reachable vertex, visiting each vertex at most once. $\text{dfs}(v)$ is as follows:

- Mark v
- For each unmarked adjacent vertex w :
 - set $\text{edgeTo}[w] = v$
 - $\text{dfs}(w)$

#	marked	edgeTo	dfs(6):
0	T	-	mark(6).
1	T	0	
2	T	1	isMarked(5)? Yes.
3	T	4	isMarked(7)? No.
4	T	5	● edgeTo[7] = 6. dfs(7).
5	T	2	
6	T	5	No more children so return.
7	T	6	
8	F	-	

Order of dfs calls: 01254367



Order of dfs returns: 3476

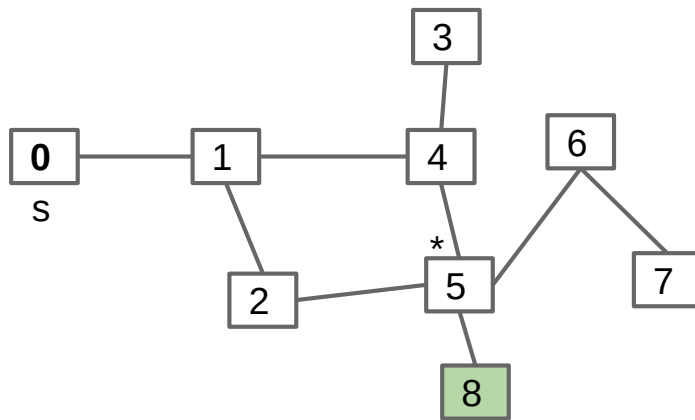
DepthFirstPaths Demo

Goal: Find a path from s to every other reachable vertex, visiting each vertex at most once. $\text{dfs}(v)$ is as follows:

- Mark v
- For each unmarked adjacent vertex w :
 - set $\text{edgeTo}[w] = v$
 - $\text{dfs}(w)$

#	marked	edgeTo	dfs(5):
0	T	-	mark(5).
1	T	0	
2	T	1	isMarked(2)? Yes.
3	T	4	isMarked(4)?
4	T	5	● $\text{edgeTo}[3] = 4$. $\text{dfs}(4)$.
5	T	2	isMarked(6)?
6	T	5	● $\text{edgeTo}[6] = 5$. $\text{dfs}(6)$.
7	T	6	isMarked(8)? No.
8	F	5	● $\text{edgeTo}[8] = 5$. $\text{dfs}(8)$

Order of dfs calls: 012543678



Order of dfs returns: 3476

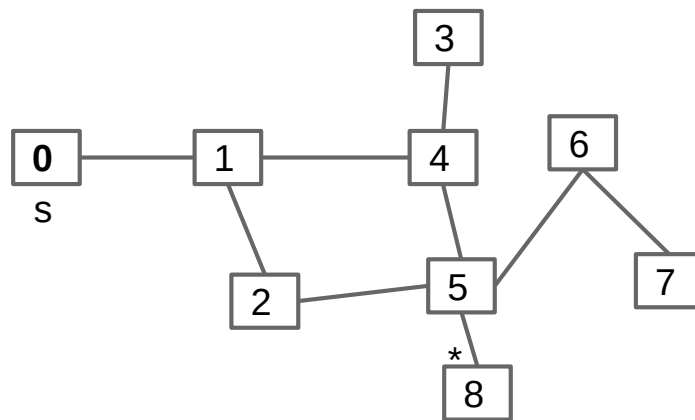
DepthFirstPaths Demo

Goal: Find a path from s to every other reachable vertex, visiting each vertex at most once. $\text{dfs}(v)$ is as follows:

- Mark v
- For each unmarked adjacent vertex w :
 - set $\text{edgeTo}[w] = v$
 - $\text{dfs}(w)$

#	marked	edgeTo	dfs(8):
0	T	-	mark(8)
1	T	0	
2	T	1	isMarked(5)? Yes.
3	T	4	
4	T	5	No more children, so return.
5	T	2	
6	T	5	
7	T	6	
8	T	5	

Order of dfs calls: 012543678



Order of dfs returns: 34768

DepthFirstPaths Demo

Goal: Find a path from s to every other reachable vertex, visiting each vertex at most once. $\text{dfs}(v)$ is as follows:

- Mark v
- For each unmarked adjacent vertex w :
 - set $\text{edgeTo}[w] = v$
 - $\text{dfs}(w)$

#	marked	edgeTo
0	T	-
1	T	0
2	T	1
3	T	4
4	T	5
5	T	2
6	T	5
7	T	6
8	T	5

$\text{dfs}(5)$:

mark(5).

isMarked(2)? Yes.

isMarked(4)?

- $\text{edgeTo}[3] = 4$. $\text{dfs}(4)$.

isMarked(6)?

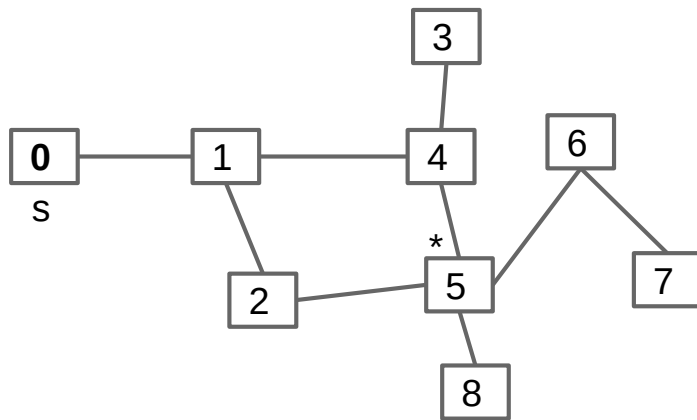
- $\text{edgeTo}[6] = 5$. $\text{dfs}(6)$.

isMarked(8)? No.

- $\text{edgeTo}[8] = 5$. $\text{dfs}(8)$

No more children, return.

Order of dfs calls: 012543678



Order of dfs returns: 347685

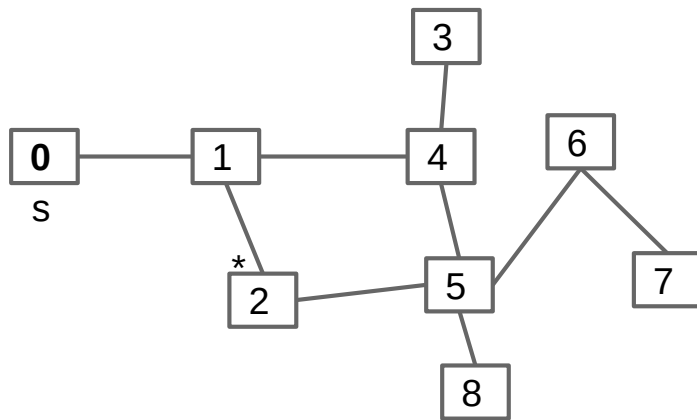
DepthFirstPaths Demo

Goal: Find a path from s to every other reachable vertex, visiting each vertex at most once. $\text{dfs}(v)$ is as follows:

- Mark v
- For each unmarked adjacent vertex w :
 - set $\text{edgeTo}[w] = v$
 - $\text{dfs}(w)$

#	marked	edgeTo	dfs(2):
0	T	-	mark(2).
1	T	0	
2	T	1	isMarked(1)? Yes.
3	T	4	isMarked(5)?
4	T	5	● edgeTo[5] = 2. dfs(5).
5	T	2	
6	T	5	No more children, so return.
7	T	6	
8	T	5	

Order of dfs calls: 012543678



Order of dfs returns: 3476852

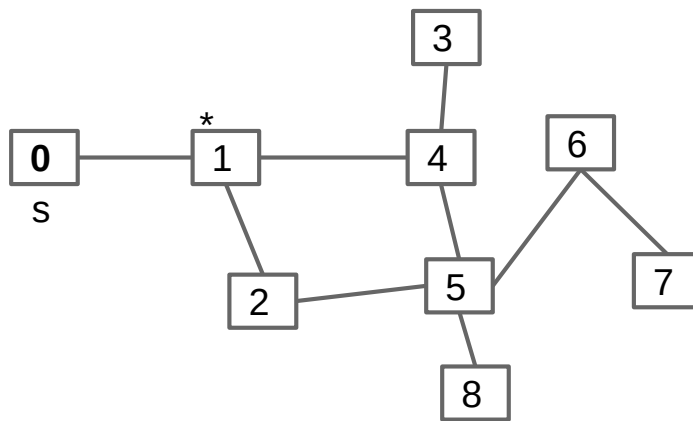
DepthFirstPaths Demo

Goal: Find a path from s to every other reachable vertex, visiting each vertex at most once. $\text{dfs}(v)$ is as follows:

- Mark v
- For each unmarked adjacent vertex w :
 - set $\text{edgeTo}[w] = v$
 - $\text{dfs}(w)$

#	marked	edgeTo	dfs(1):
0	T	-	mark(1).
1	T	0	
2	T	1	isMarked(0)? Yes.
3	T	4	isMarked(2)?
4	T	5	● edgeTo[2] = 1. dfs(2).
5	T	2	isMarked(4)? Yes.
6	T	5	
7	T	6	
8	T	5	No more children, so return.

Order of dfs calls: 012543678



Order of dfs returns: 34768521

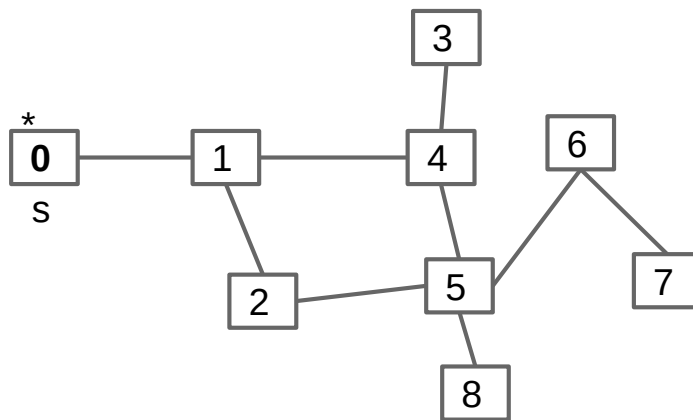
DepthFirstPaths Demo

Goal: Find a path from s to every other reachable vertex, visiting each vertex at most once. $\text{dfs}(v)$ is as follows:

- Mark v
- For each unmarked adjacent vertex w :
 - set $\text{edgeTo}[w] = v$
 - $\text{dfs}(w)$

#	marked	edgeTo	dfs(0):
0	T	-	mark(0).
1	T	0	
2	T	1	isMarked(1)? No.
3	T	4	● $\text{edgeTo}[1] = 0$. $\text{dfs}(1)$.
4	T	5	
5	T	2	No more children, so return.
6	T	5	
7	T	6	
8	T	5	

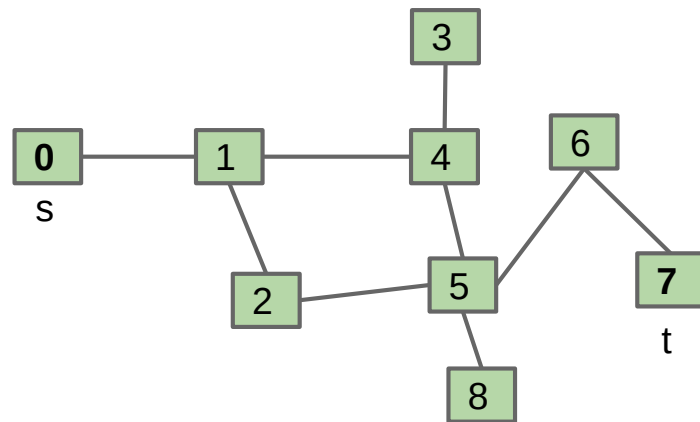
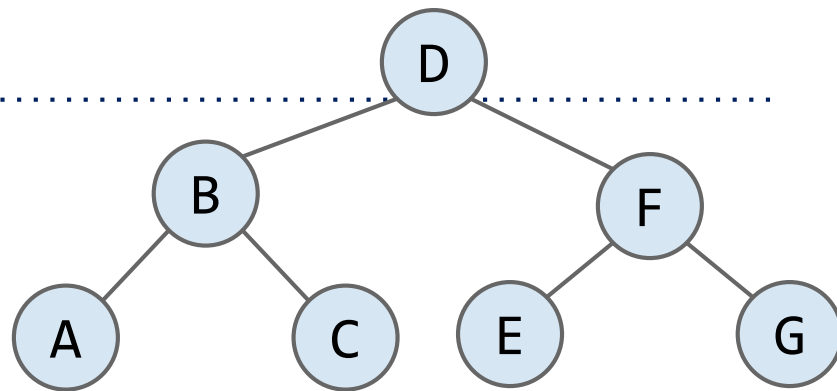
Order of dfs calls: 012543678



Order of dfs returns: 347685210

Graph Traversals

- Recall the preorder tree traversal : DBACFEG
- What we just did in DepthFirstPaths is called **DFS Preorder**
 - visit (action, print) is ***before*** DFS calls to neighbors
 - the action was setting the array edgeTo
 - edgeTo[1] was set before DFS calls to neighbors 2 and 4
 - one valid DFS preorder for this graph: 012543678
 - equivalent to the order of dfs calls

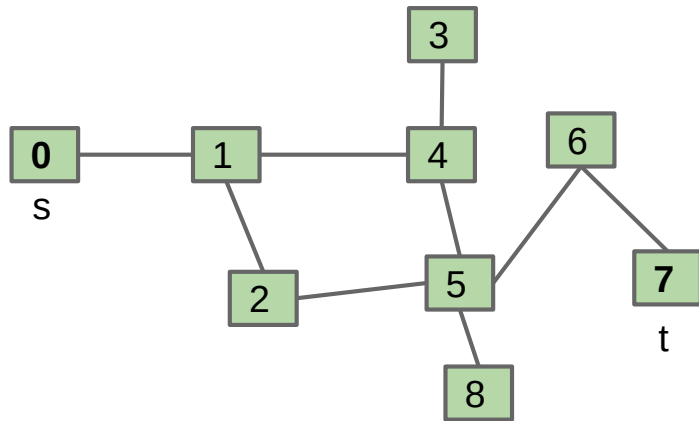


Graph Traversals

- We could also do actions in

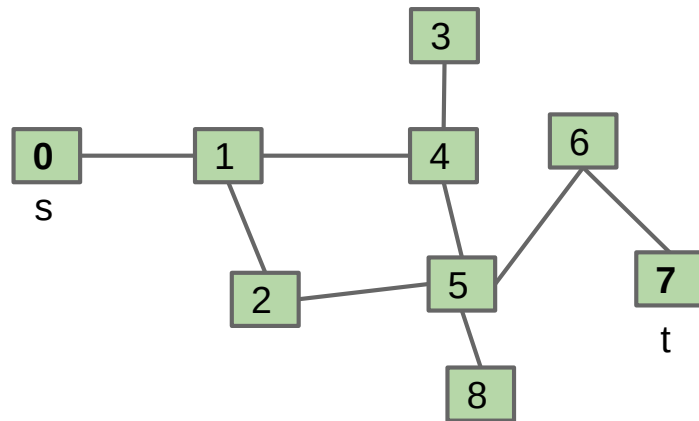
DFS Postorder

- visit (action, print) is *after* DFS calls to neighbors
- dfs(s):
 - mark(s)
 - for each unmarked neighbor n of s, dfs(n)
 - print(s)
- the result for dfs(0) would be:
347685210
- equivalent to the order of dfs returns



Shortest Path Problem

- Given a graph, find the **shortest path** from s to all other vertices
 - DFS only gives us a path, not necessarily the shortest
 - do actions before/after traversing the neighbors
 - Can we do action in the order of *distance from s* ?



Breadth First Search

- Initialize a queue with a starting vertex s , mark that vertex, (and set $dist$ to 0)
 - a **queue** is a list that has two operations: enqueue and dequeue
 - first in, first out (FIFO)
- Repeat until queue is empty:
 - remove vertex v from the front of the queue
 - for each unmarked neighbor n of v :
 - mark n
 - set $edgeTo[n] = v$ (and $distTo[n] = distTo[v] + 1$)
 - add n to end of queue



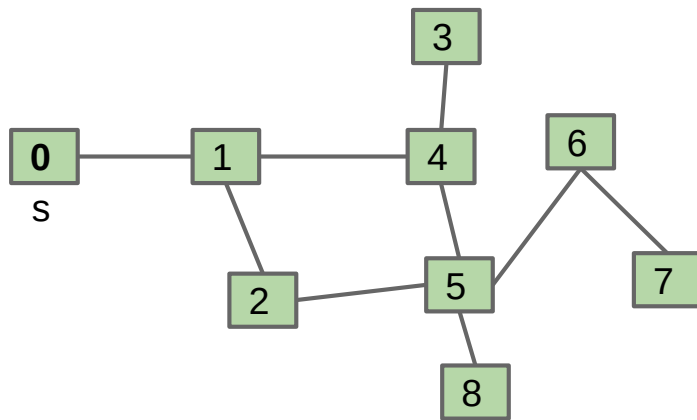
to track distance value

BreadthFirstPaths Demo

Goal: Find shortest path between s and every other vertex

- Initialize a queue with a starting vertex s , mark that vertex, set $dist$ to 0
- Repeat until queue is empty:
 - remove vertex v from fringe
 - for each unmarked neighbor n of v : mark n , add n to queue, set $edgeTo[n] = v$, set $distTo[n] = distTo[v] + 1$

#	marked	edgeTo	distTo
0	F	-	-
1	F	-	-
2	F	-	-
3	F	-	-
4	F	-	-
5	F	-	-
6	F	-	-
7	F	-	-
8	F	-	-



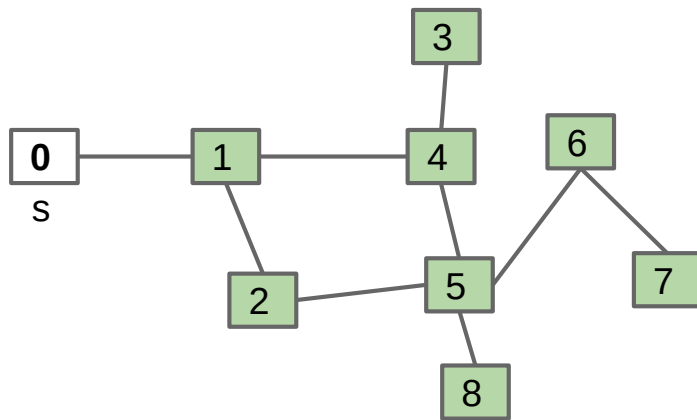
Queue: []

BreadthFirstPaths Demo

Goal: Find shortest path between s and every other vertex

- Initialize a queue with a starting vertex s , mark that vertex, set $dist$ to 0
- Repeat until queue is empty:
 - remove vertex v from fringe
 - for each unmarked neighbor n of v : mark n , add n to queue, set $edgeTo[n] = v$, set $distTo[n] = distTo[v] + 1$

#	marked	edgeTo	distTo
0	T	-	0
1	F	-	-
2	F	-	-
3	F	-	-
4	F	-	-
5	F	-	-
6	F	-	-
7	F	-	-
8	F	-	-



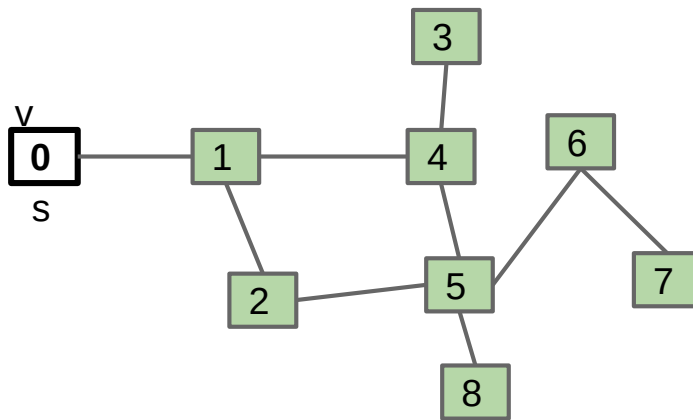
Queue: [0]

BreadthFirstPaths Demo

Goal: Find shortest path between s and every other vertex

- Initialize a queue with a starting vertex s , mark that vertex, set dist to 0
- Repeat until queue is empty:
 - remove vertex v from fringe
 - for each unmarked neighbor n of v : mark n , add n to queue, set $\text{edgeTo}[n] = v$, set $\text{distTo}[n] = \text{distTo}[v] + 1$

#	marked	edgeTo	distTo
0	T	-	0
1	F	-	-
2	F	-	-
3	F	-	-
4	F	-	-
5	F	-	-
6	F	-	-
7	F	-	-
8	F	-	-



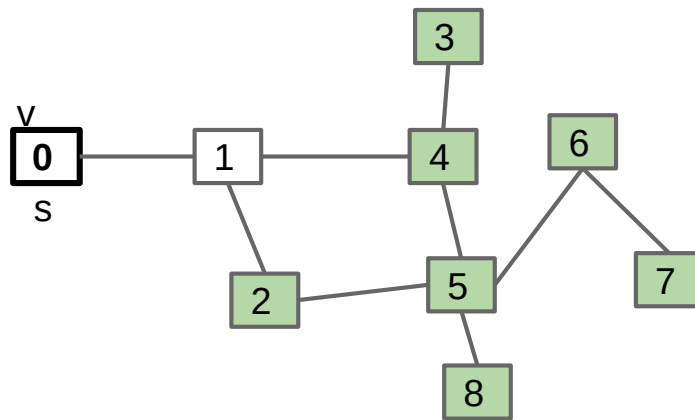
Queue: []

BreadthFirstPaths Demo

Goal: Find shortest path between s and every other vertex

- Initialize a queue with a starting vertex s , mark that vertex, set $dist$ to 0
- Repeat until queue is empty:
 - remove vertex v from fringe
 - for each unmarked neighbor n of v : mark n , add n to queue, set $edgeTo[n] = v$, set $distTo[n] = distTo[v] + 1$

#	marked	edgeTo	distTo
0	T	-	0
1	T	0	1
2	F	-	-
3	F	-	-
4	F	-	-
5	F	-	-
6	F	-	-
7	F	-	-
8	F	-	-



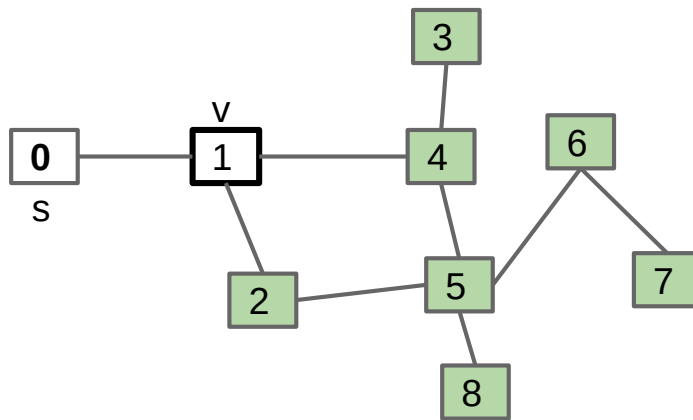
Queue: [1]

BreadthFirstPaths Demo

Goal: Find shortest path between s and every other vertex

- Initialize a queue with a starting vertex s , mark that vertex, set $dist$ to 0
- Repeat until queue is empty:
 - remove vertex v from fringe
 - for each unmarked neighbor n of v : mark n , add n to queue, set $edgeTo[n] = v$, set $distTo[n] = distTo[v] + 1$

#	marked	edgeTo	distTo
0	T	-	0
1	T	0	1
2	F	-	-
3	F	-	-
4	F	-	-
5	F	-	-
6	F	-	-
7	F	-	-
8	F	-	-



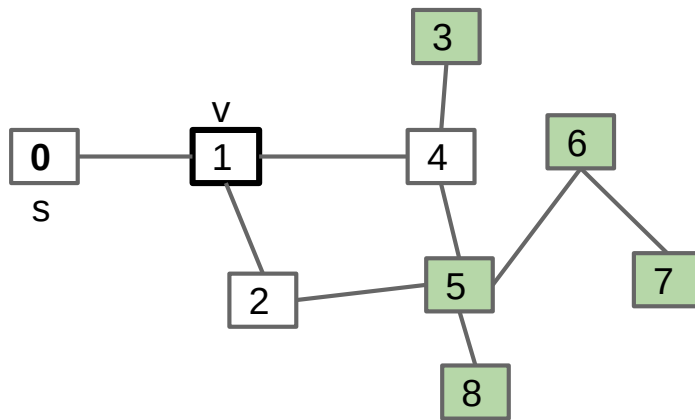
Queue: []

BreadthFirstPaths Demo

Goal: Find shortest path between s and every other vertex

- Initialize a queue with a starting vertex s , mark that vertex, set dist to 0
- Repeat until queue is empty:
 - remove vertex v from fringe
 - for each unmarked neighbor n of v : mark n , add n to queue, set $\text{edgeTo}[n] = v$, set $\text{distTo}[n] = \text{distTo}[v] + 1$

#	marked	edgeTo	distTo
0	T	-	0
1	T	0	1
2	T	1	2
3	F	-	-
4	T	1	2
5	F	-	-
6	F	-	-
7	F	-	-
8	F	-	-



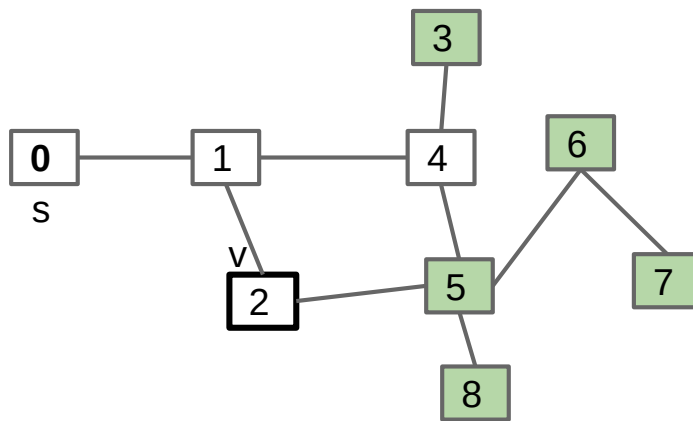
Queue: [2, 4]

BreadthFirstPaths Demo

Goal: Find shortest path between s and every other vertex

- Initialize a queue with a starting vertex s , mark that vertex, set dist to 0
- Repeat until queue is empty:
 - remove vertex v from fringe
 - for each unmarked neighbor n of v : mark n , add n to queue, set $\text{edgeTo}[n] = v$, set $\text{distTo}[n] = \text{distTo}[v] + 1$

#	marked	edgeTo	distTo
0	T	-	0
1	T	0	1
2	T	1	2
3	F	-	-
4	T	1	2
5	F	-	-
6	F	-	-
7	F	-	-
8	F	-	-



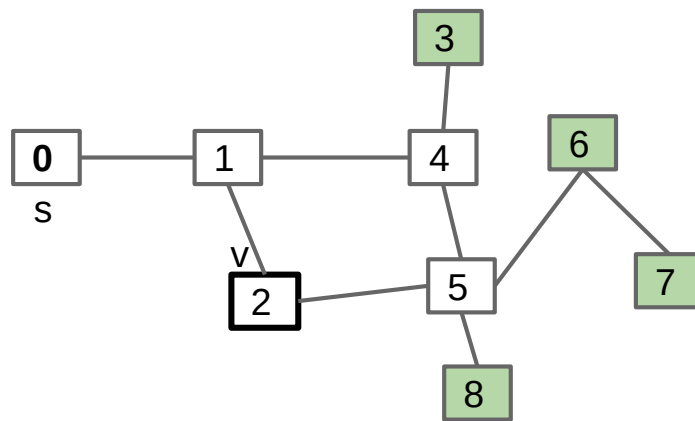
Queue: [4]

BreadthFirstPaths Demo

Goal: Find shortest path between s and every other vertex

- Initialize a queue with a starting vertex s , mark that vertex, set $dist$ to 0
- Repeat until queue is empty:
 - remove vertex v from fringe
 - for each unmarked neighbor n of v : mark n , add n to queue, set $edgeTo[n] = v$, set $distTo[n] = distTo[v] + 1$

#	marked	edgeTo	distTo
0	T	-	0
1	T	0	1
2	T	1	2
3	F	-	-
4	T	1	2
5	T	2	3
6	F	-	-
7	F	-	-
8	F	-	-



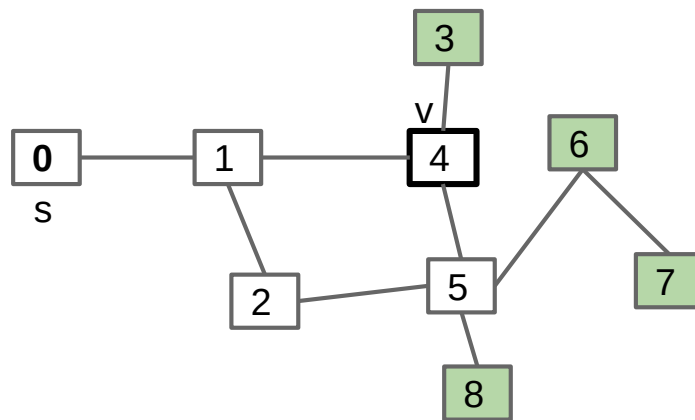
Queue: [4, 5]

BreadthFirstPaths Demo

Goal: Find shortest path between s and every other vertex

- Initialize a queue with a starting vertex s , mark that vertex, set $dist$ to 0
- Repeat until queue is empty:
 - remove vertex v from fringe
 - for each unmarked neighbor n of v : mark n , add n to queue, set $edgeTo[n] = v$, set $distTo[n] = distTo[v] + 1$

#	marked	edgeTo	distTo
0	T	-	0
1	T	0	1
2	T	1	2
3	F	-	-
4	T	1	2
5	T	2	3
6	F	-	-
7	F	-	-
8	F	-	-



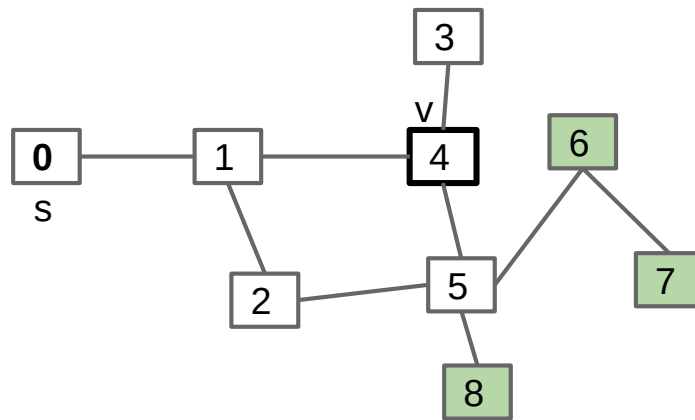
Queue: [5]

BreadthFirstPaths Demo

Goal: Find shortest path between s and every other vertex

- Initialize a queue with a starting vertex s , mark that vertex, set $dist$ to 0
- Repeat until queue is empty:
 - remove vertex v from fringe
 - for each unmarked neighbor n of v : mark n , add n to queue, set $edgeTo[n] = v$, set $distTo[n] = distTo[v] + 1$

#	marked	edgeTo	distTo
0	T	-	0
1	T	0	1
2	T	1	2
3	T	4	3
4	T	1	2
5	T	2	3
6	F	-	-
7	F	-	-
8	F	-	-



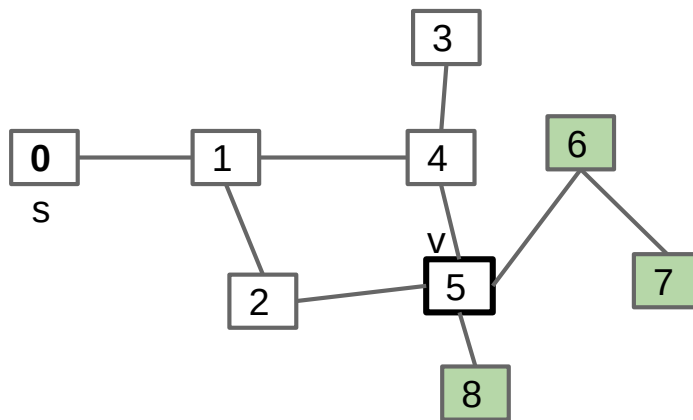
Queue: [5, 3]

BreadthFirstPaths Demo

Goal: Find shortest path between s and every other vertex

- Initialize a queue with a starting vertex s , mark that vertex, set $dist$ to 0
- Repeat until queue is empty:
 - remove vertex v from fringe
 - for each unmarked neighbor n of v : mark n , add n to queue, set $edgeTo[n] = v$, set $distTo[n] = distTo[v] + 1$

#	marked	edgeTo	distTo
0	T	-	0
1	T	0	1
2	T	1	2
3	T	4	3
4	T	1	2
5	T	2	3
6	F	-	-
7	F	-	-
8	F	-	-



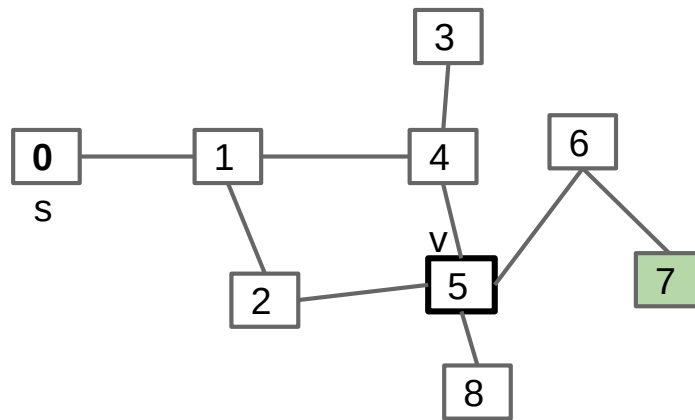
Queue: [3]

BreadthFirstPaths Demo

Goal: Find shortest path between s and every other vertex

- Initialize a queue with a starting vertex s , mark that vertex, set $dist$ to 0
- Repeat until queue is empty:
 - remove vertex v from fringe
 - for each unmarked neighbor n of v : mark n , add n to queue, set $edgeTo[n] = v$, set $distTo[n] = distTo[v] + 1$

#	marked	edgeTo	distTo
0	T	-	0
1	T	0	1
2	T	1	2
3	T	4	3
4	T	1	2
5	T	2	3
6	T	5	4
7	F	-	-
8	T	5	4



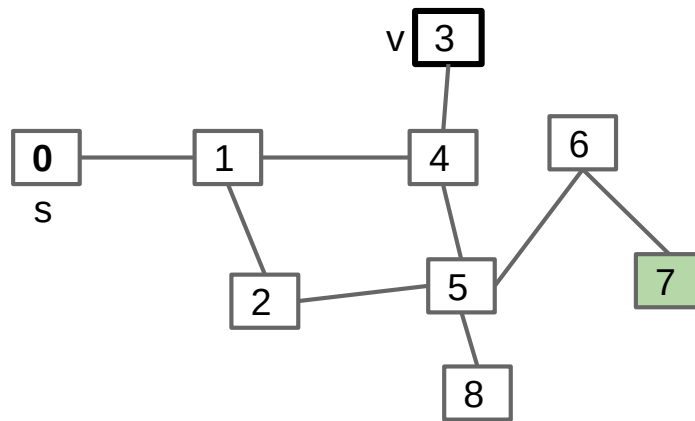
Queue: [3, 6, 8]

BreadthFirstPaths Demo

Goal: Find shortest path between s and every other vertex

- Initialize a queue with a starting vertex s , mark that vertex, set $dist$ to 0
- Repeat until queue is empty:
 - remove vertex v from fringe
 - for each unmarked neighbor n of v : mark n , add n to queue, set $edgeTo[n] = v$, set $distTo[n] = distTo[v] + 1$

#	marked	edgeTo	distTo
0	T	-	0
1	T	0	1
2	T	1	2
3	T	4	3
4	T	1	2
5	T	2	3
6	T	5	4
7	F	-	-
8	T	5	4



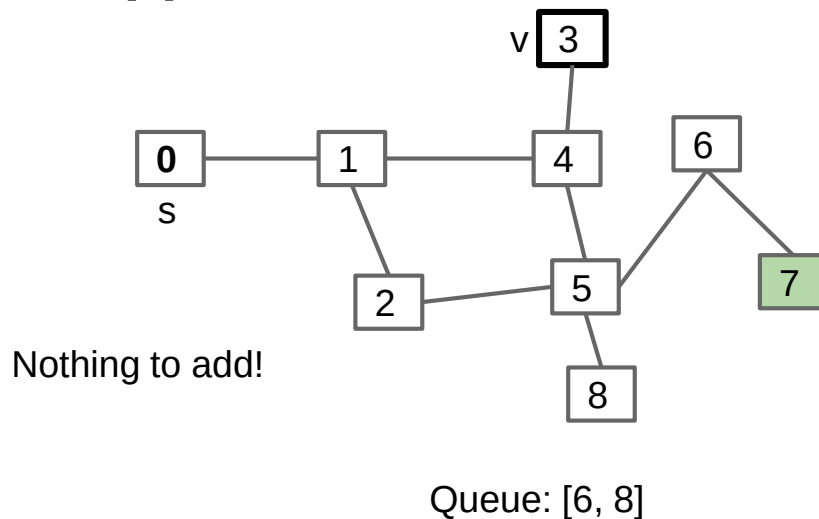
Queue: [6, 8]

BreadthFirstPaths Demo

Goal: Find shortest path between s and every other vertex

- Initialize a queue with a starting vertex s , mark that vertex, set $dist$ to 0
- Repeat until queue is empty:
 - remove vertex v from fringe
 - for each unmarked neighbor n of v : mark n , add n to queue, set $edgeTo[n] = v$, set $distTo[n] = distTo[v] + 1$

#	marked	edgeTo	distTo
0	T	-	0
1	T	0	1
2	T	1	2
3	T	4	3
4	T	1	2
5	T	2	3
6	T	5	4
7	F	-	-
8	T	5	4

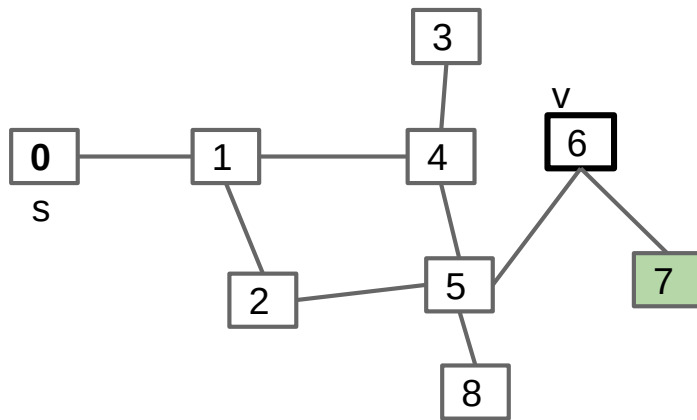


BreadthFirstPaths Demo

Goal: Find shortest path between s and every other vertex

- Initialize a queue with a starting vertex s , mark that vertex, set $dist$ to 0
- Repeat until queue is empty:
 - remove vertex v from fringe
 - for each unmarked neighbor n of v : mark n , add n to queue, set $edgeTo[n] = v$, set $distTo[n] = distTo[v] + 1$

#	marked	edgeTo	distTo
0	T	-	0
1	T	0	1
2	T	1	2
3	T	4	3
4	T	1	2
5	T	2	3
6	T	5	4
7	F	-	-
8	T	5	4



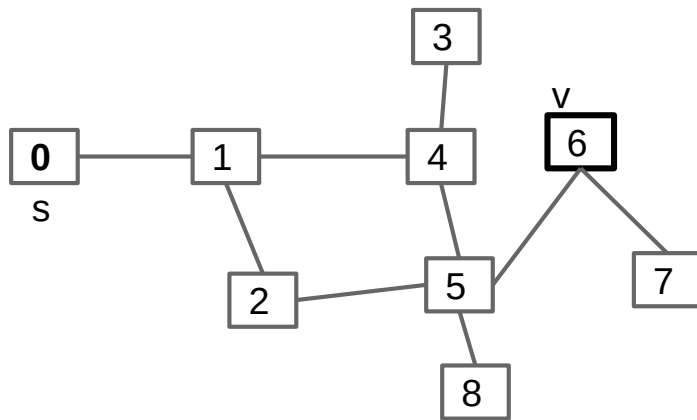
Queue: [8]

BreadthFirstPaths Demo

Goal: Find shortest path between s and every other vertex

- Initialize a queue with a starting vertex s , mark that vertex, set $dist$ to 0
- Repeat until queue is empty:
 - remove vertex v from fringe
 - for each unmarked neighbor n of v : mark n , add n to queue, set $edgeTo[n] = v$, set $distTo[n] = distTo[v] + 1$

#	marked	edgeTo	distTo
0	T	-	0
1	T	0	1
2	T	1	2
3	T	4	3
4	T	1	2
5	T	2	3
6	T	5	4
7	T	6	5
8	T	5	4



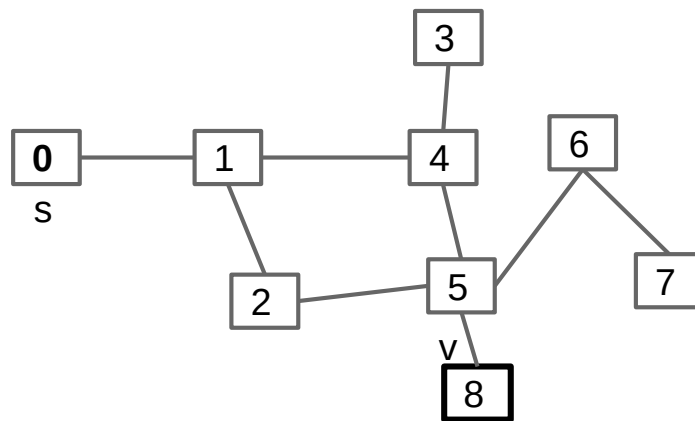
Queue: [8, 7]

BreadthFirstPaths Demo

Goal: Find shortest path between s and every other vertex

- Initialize a queue with a starting vertex s , mark that vertex, set $dist$ to 0
- Repeat until queue is empty:
 - remove vertex v from fringe
 - for each unmarked neighbor n of v : mark n , add n to queue, set $edgeTo[n] = v$, set $distTo[n] = distTo[v] + 1$

#	marked	edgeTo	distTo
0	T	-	0
1	T	0	1
2	T	1	2
3	T	4	3
4	T	1	2
5	T	2	3
6	T	5	4
7	T	6	5
8	T	5	4

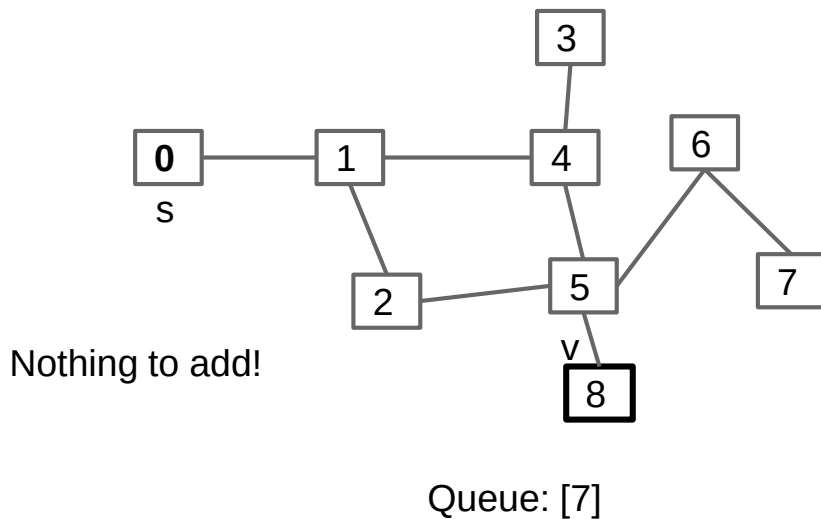


BreadthFirstPaths Demo

Goal: Find shortest path between s and every other vertex

- Initialize a queue with a starting vertex s , mark that vertex, set $dist$ to 0
- Repeat until queue is empty:
 - remove vertex v from fringe
 - for each unmarked neighbor n of v : mark n , add n to queue, set $edgeTo[n] = v$, set $distTo[n] = distTo[v] + 1$

#	marked	edgeTo	distTo
0	T	-	0
1	T	0	1
2	T	1	2
3	T	4	3
4	T	1	2
5	T	2	3
6	T	5	4
7	T	6	5
8	T	5	4

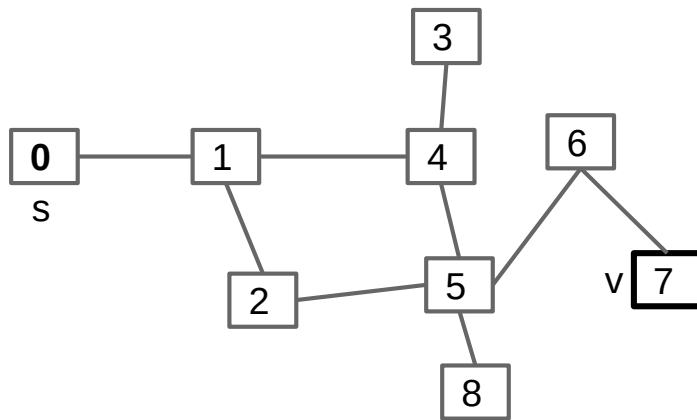


BreadthFirstPaths Demo

Goal: Find shortest path between s and every other vertex

- Initialize a queue with a starting vertex s , mark that vertex, set $dist$ to 0
- Repeat until queue is empty:
 - remove vertex v from fringe
 - for each unmarked neighbor n of v : mark n , add n to queue, set $edgeTo[n] = v$, set $distTo[n] = distTo[v] + 1$

#	marked	edgeTo	distTo
0	T	-	0
1	T	0	1
2	T	1	2
3	T	4	3
4	T	1	2
5	T	2	3
6	T	5	4
7	T	6	5
8	T	5	4



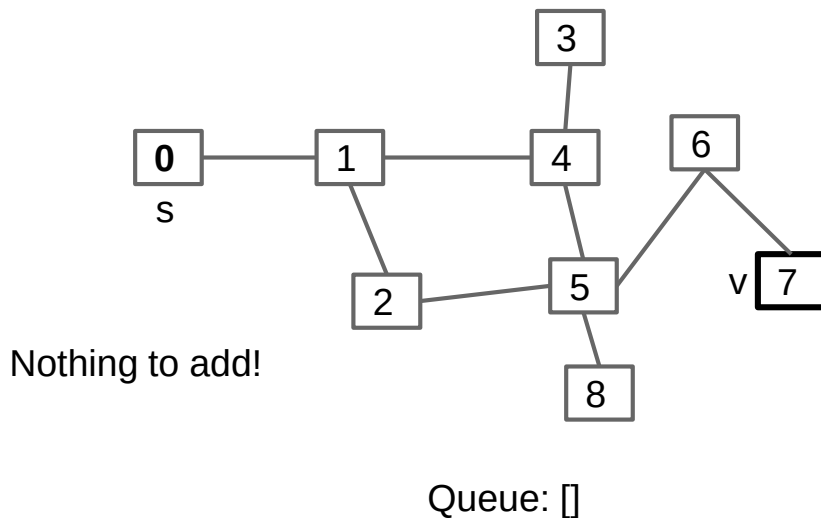
Queue: []

BreadthFirstPaths Demo

Goal: Find shortest path between s and every other vertex

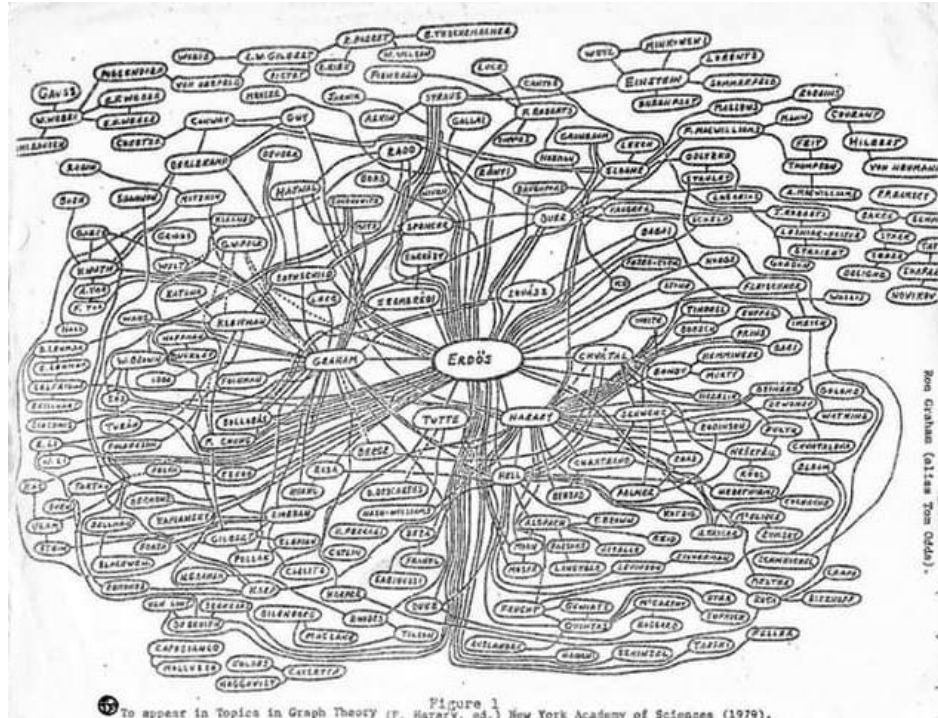
- Initialize a queue with a starting vertex s , mark that vertex, set $dist$ to 0
- Repeat until queue is empty:
 - remove vertex v from fringe
 - for each unmarked neighbor n of v : mark n , add n to queue, set $edgeTo[n] = v$, set $distTo[n] = distTo[v] + 1$

#	marked	edgeTo	distTo
0	T	-	0
1	T	0	1
2	T	1	2
3	T	4	3
4	T	1	2
5	T	2	3
6	T	5	4
7	T	6	5
8	T	5	4



Erdős Number

- The **Erdős number** describes the *collaborative distance* between researcher *Paul Erdős* and another person, measured by joint authorship of papers



Graph Implementation

- How do we implement a graph?

```
public class Graph {  
    public Graph(int V):           create empty graph with v vertices  
    public void addEdge(int v, int w): add an edge v-w  
    Iterable<Integer> adj(int v):  vertices adjacent to v  
    int V():                      number of vertices  
    int E():                      number of edges  
    ...  
}
```

- what is fixed in this implementation?
- what types of graphs are supported / not supported?

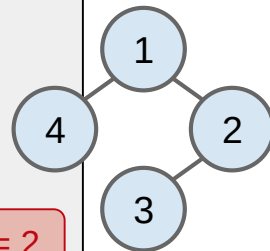
Sample Method: Degree

- Using this definition, we can write:

```
public class Graph {  
    public Graph(int V):           create empty graph with v vertices  
    public void addEdge(int v, int w): add an edge v-w  
    Iterable<Integer> adj(int v):  vertices adjacent to v  
    int V():                       number of vertices  
    int E():                       number of edges  
    ...  
}
```

- a method to compute the **degree** (#edges) of a vertex:

```
public static int degree(Graph G, int v) {  
    int degree = 0;  
    for (int w : G.adj(v)) {  
        degree += 1;  
    }  
    return degree; }  
}
```



degree(G, 2) = 2

Sample Method: Print Edges

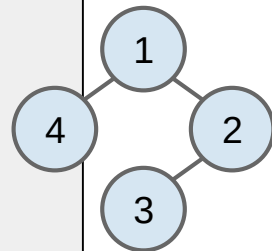
- Using this definition, we can write:

```
public class Graph {  
    public Graph(int V):           create empty graph with v vertices  
    public void addEdge(int v, int w): add an edge v-w  
    Iterable<Integer> adj(int v):  vertices adjacent to v  
    int V():                       number of vertices  
    int E():                       number of edges  
    ...  
}
```

- a method to print edges

```
1 - 2  
1 - 4  
2 - 1  
2 - 3  
3 - 2  
4 - 1
```

```
public static void print(Graph G) {  
    for (int v = 0; v < G.V(); v += 1) {  
        for (int w : G.adj(v)) {  
            System.out.println(v + "-" + w);  
        }  
    }  
}
```



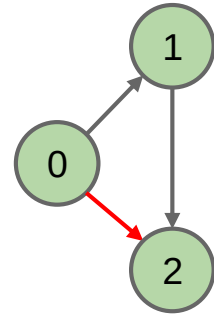
Graph Representation 1

- **Adjacency Matrix:**

2D boolean array

- Directed graph:

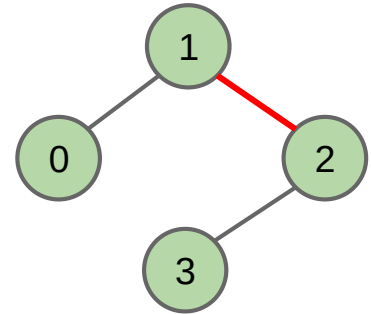
s \ t	0	1	2
0	0	1	1
1	0	0	1
2	0	0	0



- Undirected graph:

Each edge is represented twice in the matrix

v \ w	0	1	2	3
0	0	1	0	0
1	1	0	1	0
2	0	1	0	1
3	0	0	1	0



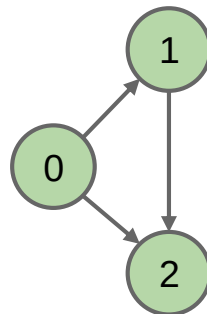
Graph Print Edges Running Time 1

- What is the running time of the print method from before if the graph uses an **adjacency matrix representation**, where V is the number of vertices, and E is the total number of edges?
 - $O(V)$
 - $O(E)$
 - $O(V + E)$
 - $O(V^2)$
 - $O(V * E)$
 - $O(E^2)$

Graph Representation 2

- **Edge Set:** collection of all edges
 - example: `HashSet<Edge>`, where each `Edge` is a pair of ints

$\{(0, 1), (0, 2), (1, 2)\}$

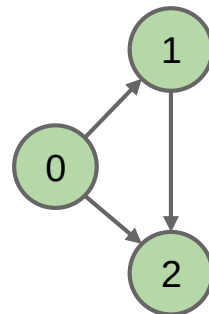
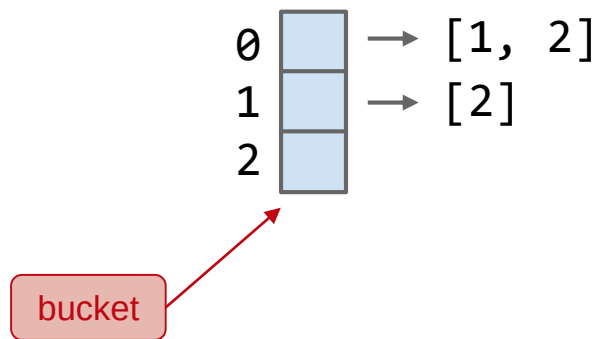


Graph Print Edges Running Time 2

- What is the running time of the print method from before if the graph uses an **edge set representation**, where V is the number of vertices, and E is the total number of edges?
 - $O(V)$
 - $O(E)$
 - $O(V + E)$
 - $O(V^2)$
 - $O(V * E)$
 - $O(E^2)$

Graph Representation 3

- **Adjacency List:** an array of lists indexed by vertex number
 - most popular approach to represent graphs



Graph Print Edges Running Time 3

- What is the running time of the print method from before if the graph uses an **adjacency list representation**, where V is the number of vertices, and E is the total number of edges?
 - $O(V)$
 - $O(E)$
 - $O(V + E)$
 - $O(V^2)$
 - $O(V * E)$
 - $O(E^2)$

Graph Representation Basic Operations

- Runtime of some basic operations for each representation:

Representation	addEdge(s, t)	for(w : adj(v))	print()	hasEdge(s, t)	space used
Adjacency matrix	$O(1)$	$O(V)$	$O(V^2)$	$O(1)$	$O(V^2)$
Edge set	$O(1)$	$O(E)$	$O(E)$	$O(E)$	$O(E)$
Adjacency list	$O(1)$	$O(V)$	$O(V+E)$	$O(\text{degree}(v))$	$O(V+E)$

- In practice, adjacency lists are most common
 - many graph algorithms rely on $\text{adj}(v)$
 - most graph algorithms are sparse (not too many edges in each bucket)

Undirected Graph Implementation

```
public class Graph {  
    private final int V;  
    private List<Integer>[] adj;  
  
    public Graph(int V) {  
        this.V = V;  
        adj = (List<Integer>[]) new ArrayList[V];  
        for (int v = 0; v < V; v++) {  
            adj[v] = new ArrayList<Integer>();  
        }  
    }  
  
    public void addEdge(int v, int w) {  
        adj[v].add(w);    adj[w].add(v);  
    }  
  
    public Iterable<Integer> adj(int v) {  
        return adj[v];  
    }  
}
```

array of list of integers

add to both buckets

Graph Algorithm Implementation

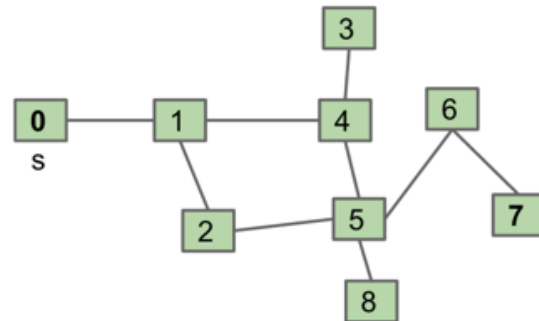
- A common design pattern in graph algorithms is that we decouple type from processing algorithm:
 - create a graph object
 - pass the graph to a graph-processing method (or constructor) in a client class
 - query the client class for information
- So in our DFS/BFS Paths case, we create a Paths class:

we don't add a new method to our Graph class

```
public class Paths {  
    public Paths(Graph G, int s):    Find all paths from G  
    boolean hasPathTo(int v):       is there a path from s to v?  
    Iterable<Integer> pathTo(int v): path from s to v (if any)  
}
```

Example Usage

- An example of using the Paths class:
 - `Paths P = new Paths(G, 0);`
 - `P.hasPathTo(3);` `// returns true`
 - `P.pathTo(3);` `// returns {0, 1, 4, 3}`



```
public class Paths {  
    public Paths(Graph G, int s):      Find all paths from G  
    boolean hasPathTo(int v):      is there a path from s to v?  
    Iterable<Integer> pathTo(int v): path from s to v (if any)  
}
```

DepthFirstPaths Recursive Implementation (1)

```
public class DepthFirstPaths {  
    private boolean[] marked;  
    private int[] edgeTo;  
    private int s;  
  
    public DepthFirstPaths(Graph G, int s) {  
        ...  
        dfs(G, s);  
    }  
    private void dfs(Graph G, int v) {  
        marked[v] = true;  
        for (int w : G.adj(v)) {  
            if (!marked[w]) {  
                edgeTo[w] = v;  
                dfs(G, w);  
            }  
        }  
    }  
}
```

marked[v] is true iff v connected to s

edgeTo[v] is previous vertex on path from s to v

data structure initialization

find vertices connected to s

DepthFirstPaths Recursive Implementation (2)

...

```
public Iterable<Integer> pathTo(int v) {  
    if (!hasPathTo(v)) return null;  
    List<Integer> path = new ArrayList<>();  
    for (int x = v; x != s; x = edgeTo[x]) {  
        path.add(x);  
    }  
    path.add(s);  
    Collections.reverse(path);  
    return path;  
}
```

```
public boolean hasPathTo(int v) {  
    return marked[v];  
}
```

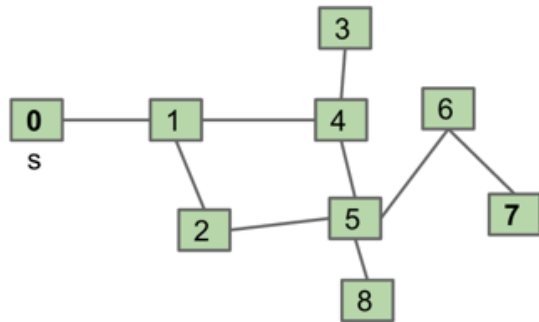
...

```
}
```

DepthFirstPaths Runtime Analysis

```
public class DepthFirstPaths {  
    private boolean[] marked;  
    private int[] edgeTo;  
    private int s;  
  
    public DepthFirstPaths(Graph G, int s) {  
        ...  
        dfs(G, s);  
    }  
    private void dfs(Graph G, int v) {  
        marked[v] = true;  
        for (int w : G.adj(v)) {  
            if (!marked[w]) {  
                edgeTo[w] = v;  
                dfs(G, w);  
            }  
        }  
    }  
    ...  
}
```

Assume graph representation is
adjacency list



vertex visits (no more than V calls)
and initialisations

edge considerations
(no more than $2E$ calls)

We sum:

- number of dfs calls: each vertex is visited at most once $O(V)$
- marked[w] checks: each edge is considered at most twice $O(E)$

so the **runtime** is **$O(V + E)$**

BreadthFirstPaths Implementation

```
public class BreadthFirstPaths {  
    private boolean[] marked;  
    private int[] edgeTo;  
    ...  
  
    private void bfs(Graph G, int s) {  
        Queue<Integer> queue =  
            new Queue<Integer>();  
        queue.enqueue(s);  
        marked[s] = true;  
        while (!queue.isEmpty()) {  
            int v = queue.dequeue();  
            for (int w : G.adj(v)) {  
                if (!marked[w]) {  
                    queue.enqueue(w);  
                    marked[w] = true;  
                    edgeTo[w] = v;  
                }  
            }  
        }  
    }  
    ...  
}
```

marked[v] is true iff v connected to s

edgeTo[v] is previous vertex on path from s to v

set up starting vertex

for freshly dequeued vertex v,
for each neighbor that is unmarked:
- enqueue that neighbor to the queue
- mark it
- set its edgeTo to v

runtime is also $O(V + E)$ with similar arguments

Thank you for your attention !

- In this lecture, you have learned:
 - about tree traversals: preorder, inorder, postorder
 - about graph data structures: adjacency matrix, edge set, adjacency list
 - graph traversals: DFS preorder, DFS postorder, BFS
 - by performing actions (print, set an array, etc) during a graph traversal, you can solve problems like s-t connectivity or path finding
 - implement and analyze finding paths using DFS and BFS in $O(V + E)$ time
- Please continue to work on **Exercise 9**